LangChain and LlamaIndex Projects Lab Book:

Hooking Large Language Models Up to the Real World

by Mark Watson



LangChain and LlamaIndex Projects Lab Book: Hooking Large Language Models Up to the Real World

Using GPT-4, ChatGPT, Hugging Face, and local Ollama Models in Applications.

Mark Watson

This book is available at https://leanpub.com/langchain

This version was published on 2025-05-06



This is a Leanpub book. Leanpub empowers authors and publishers with the Lean Publishing process. Lean Publishing is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Also By Mark Watson

Ollama in Action: Building Safe, Private AI with LLMs, Function Calling and Agents
Practical AI with Google: A Solo Knowledge Worker's Guide to Gemini, AI Studio, and LLM APIs
Safe For Humans AI
Practical Artificial Intelligence Development With Racket
Practical Artificial Intelligence Programming With Clojure
Artificial Intelligence Using Swift
A Lisp Programmer Living in Python-Land: The Hy Programming Language
Haskell Tutorial and Cookbook
Loving Common Lisp, or the Savvy Programmer's Secret Weapon
Practical Artificial Intelligence Programming With Java

Contents

Preface **1**
Requests from the Author 1
Newer Commercial LangChain Integrations That Are Not Covered In This Book 1
Deprecated Book Examples 2
LLM Hallucinations and RAG, Summarization, Structured Data Conversion, and Fact/Relationship Extraction LLM Applications 2
Comparing LangChain and LlamaIndex 3
About the Author 4
Book Cover 4
Acknowledgements 4
Requirements for Running and Modifying Book Examples 5
Issues and Workarounds for Using the Material in this Book 6

Large Language Model Overview **7**
Technological Change is Increasing at an Exponential Rate 7
What LLMs Are and What They Are Not 8
Big Tech Businesses vs. Small Startups Using Large Language Models 9

Getting Started With LangChain **11**
Installing Necessary Packages 11
Creating a New LangChain Project 12
Basic Usage and Examples 12
Creating Embeddings 15
Using LangChain Vector Stores to Query Documents: a Simple RAG Application 16
Example Using LangChain Integrations: Using Server APIs for Google Search 19
OpenAI Model GPT-4o Example 19
LangChain Overview Wrap Up 20

Overview of LlamaIndex **22**
Using LlamaIndex for Question Answering from a Web Site 23
LlamaIndex Case Study Wrap Up 25

Extraction of Facts and Relationships from Text Data **26**
Key Capabilities of LLMs for Fact and Relationship Extraction 26
Techniques and Approaches 26

Benefits . . . 27
Applications . . . 27
Example Prompts for Getting Information About a Person from Text and Generating JSON 27
Example Code . . . 29

Using LLMs to Summarize Text . . . 31
Example Prompt . . . 31
Code Example . . . 31

LLM Techniques for Structured Data Conversion . . . 33
Example Prompt for Converting CSV Files to JSON . . . 33
Example Code for Converting CSV Files to JSON . . . 34

Retrieval Augmented Generation (RAG) Applications . . . 36
Simple RAG Example Using LlamaIndex . . . 37
RAG With Reranking Example . . . 39
RAG on CSV Spreadsheet Files . . . 41

Using Google's Knowledge Graph APIs With LangChain . . . 43
Setting Up To Access Google Knowledge Graph APIs . . . 43

Using DBPedia and WikiData as Knowledge Sources . . . 48
Using DBPedia as a Data Source . . . 49
Using Wikidata as a Data Source . . . 53

Using LLMs To Organize Information in Our Google Drives . . . 57
Setting Up Requirements. . . . 58
Write Utility To Fetch All Text Files From Top Level Google Drive Folder . . . 59
Generate Vector Indices for Files in Specific Google Drive Directories . . . 61
Google Drive Example Wrap Up . . . 62

Natural Language SQLite Database Queries With LangChain . . . 63
Natural Language Database Query Wrap Up . . . 65

Examples Using Hugging Face Open Source Models . . . 66
Using LangChain as a Wrapper for Hugging Face Prediction Model APIs . . . 66
Creating a Custom LlamaIndex Hugging Face LLM Wrapper Class That Runs on Your Laptop 67

Running Local LLMs Using Llama.cpp and LangChain . . . 71
Installing Llama.cpp with a Llama2-13b-orca Model . . . 71
Python Example . . . 71

Running Local LLMs Using Ollama . . . 74
Simple Use of a local Mistral Model Using LangChain . . . 74
Minimal Example Using Ollama with the Mistral Open Model for Retrieval Augmented Queries Against Local Documents . . . 76

Wrap Up for Running Local LLMs Using Ollama . . . 78

Using Large Language Models to Write Recipes . . . 79
Preparing Recipe Data . . . 79
A Prediction Model Using the OpenAI text-embedding-3-large Model . . . 80
Cooking Recipe Generation Wrap Up . . . 83

LangChain Agents . . . 84
Overview of LangChain Tools . . . 84
Overview of ReAct Library for Implementing Reading in LMS Applications . . . 85
LangChain Agent Tool Example Using DBPedia SPARQL Queries . . . 86
Another React Agent Tool Use Example: Search and Math Expressions . . . 92
LangChain Agent Tools Wrap Up . . . 95

Multi-prompt Search using LLMs, the Duckduckgo Search API, and Local Ollama Models 96
Example 1: "Write a business plan for a new startup using LLMs and expertise in medical billing." . . . 96
Example 2: "Common Lisp and Deep Learning consultant" . . . 97
Example Code for Multi-prompt Search using LLMs, the Duckduckgo Search API, and Local Ollama Models . . . 98

More Useful Libraries for Working with Unstructured Text Data . . . 100
EmbedChain Wrapper for LangChain Simplifies Application Development . . . 100
Kor Library . . . 101

Book Wrap Up . . . 104

Preface

I have been working in the field of artificial intelligence since 1982 and without a doubt Large Language Models (LLMs) like Genini 2.5 Pro and o3 represent the greatest advance in practical AI technology that I have experienced. Infrastructure projects like LangChain and LlamaIndex make it simpler to use LLMs and provide some level of abstraction to facilitate switching between LLMs. This book will use the LangChain[1] and LlamaIndex[2] projects along with the OpenAI GPT-4o APIs, and local models run on your computer using Ollama to solve a series of interesting problems.

As I write this new edition in May 2025, I mostly run local LLMs but I still use OpenAI APIs, as well as well as APIs from Anthropic for the Claude 3.7 model and APIs from the French company Mistral. Most of the newest examples are in the chapter on RAG and agents.

Harrison Chase started the LangChain project in October 2022 and as I wrote the first edition of this book in February 2023, the GitHub repository for LangChain 171 contributors and as I write this new edition LangChain has 2100+ contributors on GitHub. Jerry Liu started the GPT Index project (recently renamed to LlamaIndex) at the end of 2022 and the GitHub Repository for LlamaIndex https://github.com/jerryjliu/gpt_index[3] currently has 54 contributors.

The GitHub repository for examples in this book is https://github.com/mark-watson/langchain-book-examples.git. Please note that I usually update the code in the examples repository fairly frequently for library version updates, etc.

While the documentation and examples online for LangChain and LlamaIndex are excellent, I am still motivated to write this book to solve interesting problems that I like to work on involving information retrieval, natural language processing (NLP), dialog agents, and the semantic web/linked data fields. I hope that you, dear reader, will be delighted with these examples and that at least some of them will inspire your future projects.

Requests from the Author

This book will always be available to read free online at https://leanpub.com/langchain/read.

That said, I appreciate it when readers purchase my books because the income enables me to spend more time writing.

Hire the Author as a Consultant

I am available for short consulting projects. Please see https://markwatson.com.

[1] https://github.com/langchain-ai/langchain
[2] https://github.com/run-llama/llama_index
[3] https://github.com/run-llama/llama_index

Newer Commercial LangChain Integrations That Are Not Covered In This Book

LangChain has increasingly integrated proprietary services and developed features dependent on commercial APIs. Some of these services include:

- LangSmith: A commercial platform tied to LangChain that focuses on debugging, logging, and testing. It helps manage and improve prompt artifacts, and integrates with the LangChain Hub.
- LangChain Hub: This hub is designed as a repository where developers can share prompts, chains, agents, and more. It allows collaboration on LLM artifacts, making it easier for teams to build and improve applications. It supports private, organization-specific repositories alongside the public ones. Users can interact with prompts in a playground using their own API keys (for services like OpenAI or Anthropic) and can download or upload new artifacts using an SDK. LangChain is also extending this hub to include support for chains and agents

I limit the examples in this book to the core LangChain library. Most of the examples have been updated to LangChain version 0.3.25

Deprecated Book Examples

As I add new examples to this book I sometimes remove older examples that no longer seem relevant. The book text can be found in the GitHub repository for the code examples. Check out https://github.com/mark-watson/langchain-book-examples/tree/main/CHAPTERS_and_CODE_no_longer_in-book.

LLM Hallucinations and RAG, Summarization, Structured Data Conversion, and Fact/Relationship Extraction LLM Applications

When we chat with LLMs and rely on the innate information they provide the results often contain so-called "hallucinations" that can occur when a model does not know an answer so it generates plausible sounding text. Many of the examples in this book are Retrieval Augmented Generation (RAG) style applications. RAG applications provide context text with chat prompts or queries so LLMs prioritize using the provided information to answer questions. The latest Google Gemini model has an input context size of one million tokens and even smaller models you can run on your own computer using tools like Ollama[4] often support up to 128K context size.

[4] https://ollama.ai

Beyond RAG, other effective LLM applications effective at minimizing hallucinations include text summarization, structured data conversion and extraction, and fact/relationship extraction. In text summarization, LLMs condense lengthy documents into concise summaries, focusing on core information. Structured data conversion and extraction involve transforming unstructured text into organized formats or pinpointing specific information. Fact/relationship extraction allows LLMs to identify and understand key connections within text, making them less prone to misinterpretation and hallucinations.

Comparing LangChain and LlamaIndex

Since I wrote the earlier editions of this book, LangChain has matured into a comprehensive platform—spanning open-source libraries and the managed agent orchestration service LangGraph—enabling modular pipelines, rich agent frameworks, and production-grade deployment tools. Meanwhile, LlamaIndex has evolved into an enterprise-focused suite (branded as LlamaCloud) offering GenAI-native parsing, schema-driven data extraction, knowledge management, and Agentic Document Workflows (ADW) for orchestrating complex, multi-step document processes. Choose LangChain when you need end-to-end flexibility in chaining, agents, and callback instrumentation with a clear path to production via LangGraph; choose LlamaIndex when your priority is high-performance indexing, advanced parsing, and agentic workflows over large enterprise datasets.

LangChain

- Composability & Chains: Exposes primitives for prompts, chains, agents, and callbacks, allowing developers to stitch together LLM calls, embeddings, and tools into custom pipelines.
- Agentic Orchestration (LangGraph): Introduced LangGraph in January 2024 to deploy and manage agents at scale, define tool-use policies, and monitor runtime performance.
- Extensive API Reference: The Python API catalogs 200+ classes (e.g., AgentExecutor, BaseSingleActionAgent) and offers deep support for multi-action agents, memory modules, and output parsers.
- Documentation Refresh: In May 2024, LangChain released v0.2 with versioned docs, a flatter structure, and consolidated tutorials, how-to guides, conceptual overviews, and API references.

LlamaIndex

- GenAI-Native Parsing: LlamaParse handles PDFs, tables, diagrams, and multimodal content, automatically chunking and formatting for optimal LLM ingestion.
- Schema-Driven Extraction: Provides a data-extraction engine to define output schemas, transforming unstructured documents into structured JSON or tabular records.
- Knowledge Management: Connectors to databases, file systems, and cloud stores feed into an indexed knowledge base accessible via vector embeddings or keyword search.
- Agentic Document Workflows (ADW): Introduced early 2025, ADW maintains state across steps, applies business rules, coordinates components, and executes actions based on document content.

Capability	LangChain	LlamaIndex
Prompting & Chains	Highly customizable, chain-of-thought, branching, and callback hooks (LangChain Concepts[5])	Limited to indexed data retrieval plus templating
Embedding & Vector Stores	Built-in support for common stores (Chroma, Pinecone, Weaviate) (LangChain How-To[6])	Optimized local and cloud vector indices
Agents & Tools	Multi-action agents, toolkits, memory modules (LangChain API Reference[7])	Agent framework focused on knowledge assistants
Data Ingestion	Manual chunking or external utilities	Automated parsing + schema extraction
Production Deployment	LangGraph platform for scaling agents	LlamaCloud managed service

About the Author

I have written over 20 books, I have over 50 US patents, and I have worked at interesting companies like Google, Capital One, SAIC, Mind AI, and others. You can find links for reading most of my recent books free on my web site https://markwatson.com. If I had to summarize my career the short take would be that I have had a lot of fun and enjoyed my work. I hope that what you learn here will be both enjoyable and help you in your work.

If you would like to support my work please consider purchasing my books on Leanpub[8] and star my git repositories that you find useful on GitHub[9]. You can also interact with me on social media on Mastodon[10] and Twitter[11]. I am also available as a consultant: https://markwatson.com.

Book Cover

I live in Sedona, Arizona. I took the book cover photo in January 2023 from the street that I live on.

[8] https://leanpub.com/u/markwatson
[9] https://github.com/mark-watson?tab=repositories&q=&type=public
[10] https://mastodon.social/@mark_watson
[11] https://twitter.com/mark_l_watson

Acknowledgements

This picture shows me and my wife Carol who helps me with book production and editing.



Figure 1. Mark and Carol Watson

I would also like to thank the following readers who reported errors or typos in this book: Armando Flores, Peter Solimine, and David Rupp.

Requirements for Running and Modifying Book Examples

I show full source code and a fair amount of example output for each book example so if you don't want to get access to some of the following APIs then you can still read along in the book.

To use OpenAI's GPT-3 and ChatGPT models you will need to sign up for an API key (free tier is OK) at https://openai.com/api/ and set the environment variable **OPENAI_API_KEY** to your key value.

You will need to get an API key for examples using **Google's Knowledge Graph APIs**.

Reference: Google Knowledge Graph APIs[12].

The example programs using Google's Knowledge Graph APIs assume that you have the file **~/.google_api_key** in your home directory that contains your key from https://console.cloud.google.com/apis.

You will need to install **SerpApi for examples integrating web search**. Please reference: PyPi project page[13].

You can sign up for a free non-commercial 100 searches/month account with an email address and phone number at https://serpapi.com/users/welcome.

You will also need Zapier[14] account for the GMail and Google Calendar examples.

After reading though this book, you can review the website LangChainHub[15] which contains prompts, chains and agents that are useful for building LLM applications.

Issues and Workarounds for Using the Material in this Book

The libraries that I use in this book are frequently updated and sometimes the documentation or code links change, invalidating links in this book. I will try to keep everything up to date. Please report broken links to me.

In some cases you will need to use specific versions or libraries for some of the code examples.

Because the Python code listings use colorized text you may find that copying code from this eBook may drop space characters. All of the code listings are in the GitHub repository for this book so you should clone the repository to experiment with the example code.

[12] https://cloud.google.com/enterprise-knowledge-graph/docs/search-api
[13] https://pypi.org/project/google-search-results/
[14] https://zapier.com
[15] https://github.com/hwchase17/langchain-hub

Large Language Model Overview

Large language models[1] are a subset of artificial intelligence that use deep learning and neural networks to process natural language. Transformers[2] are a type of neural network architecture that can learn context in sequential data using self-attention mechanisms. They were introduced in 2017 by a team at Google Brain and have become popular for LLM research. Some older examples of transformer-based[3] LLMs are BERT, GPT-3, T5 and Megatron-LM[4].

The main points we will discuss in this book are:

- LLMs are deep learning algorithms that can understand and generate natural language based on massive datasets.
- LLMs use techniques such as self-attention, masking, and fine-tuning to learn complex patterns and relationships in language. LLMs can understand and generate natural language because they use transformer models, which are a type of neural network that can process sequential data such as text using attention mechanisms. Attention mechanisms allow the model to focus on relevant parts of the input and output sequences while ignoring irrelevant ones.
- LLMs can perform various natural language processing (NLP) and natural language generation (NLG) tasks, such as summarization, translation, prediction, classification, and question answering.
- Even though LLMs were initially developed for NLP applications, LLMs have also shown potential in other domains such as computer vision and computational biology by leveraging their generalizable knowledge and transfer learning abilities.

BERT models[5] are one of the first types of transformer models that were widely used. BERT was developed by Google AI Language in 2018. BERT models are a family of masked language models that use transformer architecture to learn bidirectional representations of natural language. BERT models can understand the meaning of ambiguous words by using the surrounding text as context. The "magic trick" here is that training data comes almost free because in masking models, you programmatically chose random words, replace them with a missing word token, and the model is trained to predict the missing words. This process is repeated with massive amounts of training data from the web, books, etc.

Here are some "papers with code" links for BERT (links are for code, paper links in the code repositories):

- https://github.com/allenai/scibert
- https://github.com/google-research/bert

[1] https://blogs.nvidia.com/blog/2022/10/10/llms-ai-horizon/
[2] https://www.linkedin.com/pulse/chatgpt-tip-iceberg-paul-golding
[3] https://factored.ai/transformer-based-language-models/
[4] https://en.wikipedia.org/wiki/Transformer_(machine_learning_model)
[5] https://en.wikipedia.org/wiki/BERT_(Language_model)

Technological Change is Increasing at an Exponential Rate

When I wrote the first edition of this book it was difficult to run LLMs locally on my own computers. Now in May 2025, I can use Ollama to run very useful models on the old M1 8G MacBook I am writing this on:

```
$ ollama list
NAME                       ID              SIZE      MODIFIED
phi4-reasoning:plus        f0ad3edce8e4    11 GB     4 days ago
qwen3:8b                   e4b5fd7f8af0    5.2 GB    6 days ago
qwen3:30b                  2ee832bc15b5    18 GB     6 days ago
gemma3:12b-it-qat          5d4fa005e7bb    8.9 GB    2 weeks ago
gemma3:4b-it-qat           d01ad0579247    4.0 GB    2 weeks ago
gemma3:27b-it-qat          29eb0b9aeda3    18 GB     2 weeks ago
openthinker:latest         4e61774f7d1c    4.7 GB    3 weeks ago
deepcoder:latest           12bdda054d23    9.0 GB    3 weeks ago
granite3-dense:latest      5c2e6f3112f4    1.6 GB    3 weeks ago
nomic-embed-text:latest    0a109f422b47    274 MB    3 weeks ago
qwen2.5:14b                7cdf5a0187d5    9.0 GB    3 weeks ago
qwq:latest                 38ee5094e51e    19 GB     3 weeks ago
cogito:32b                 0b4aab772f57    19 GB     3 weeks ago
deepseek-r1:32b            38056bbcbb2d    19 GB     3 weeks ago
mistral-small:latest       8039dd90c113    14 GB     3 weeks ago
phi4-mini:latest           78fad5d182a7    2.5 GB    3 weeks ago
reader-lm:latest           33da2b9e0afe    934 MB    3 weeks ago
deepseek-r1:8b             28f8fd6cdc67    4.9 GB    3 weeks ago
llama3.2:1b                baf6a787fdff    1.3 GB    3 weeks ago
smollm2:latest             cef4a1e09247    1.8 GB    3 weeks ago
unsloth_AZ_1B:latest       0b3006d8395a    807 MB    3 weeks ago
llama3.2:latest            a80c4f17acd5    2.0 GB    3 weeks ago
llava:7b                   8dd30f6b0cb1    4.7 GB    3 weeks ago
```

The good news is that techniques you learn now for incorporating LLMs into your own applications and you increased knowledge of and ease of writing effective prompts for LLMs will be useful even as models become more powerful.

What LLMs Are and What They Are Not

Large Language Models are text predictors. Given a prompt, or context text and a prompt or question, an LLM predicts a highly likely text completion. As human beings we have a tendency to

ascribe deep intelligence and world knowledge to LLMs. I try to avoid this misconception. A year ago I asked ChatGPT to write a poem about my pet parrot escaping out the window in the style of poet Elizabeth Bishop. When an friend asked that ChatGPT rewrite the poem in the style of more modern poet Billy Collins we both were surprised how closely it mimicked the styles of both poets. Surely this must be some deep form of intelligence, right? No, this phenomenon is text prediction on a model trained on most books and most web content.

LLMs compress knowledge of language and some knowledge of the world into a compact representation. Clever software developers can certainly build useful and interesting systems using LLMs and this is the main topic of this book. My hope is that by experimenting with writing prompts, learning the differences between available models, and practicing applying LLMs to transform textual data that you will develop your own good ideas and build your own applications that you and other people find useful.

Big Tech Businesses vs. Small Startups Using Large Language Models

Both Microsoft and Google play both sides of this business game: they want to sell cloud LLM services to developers and small startup companies and they would also like to achieve lock-in for their consumer services like Office 365, Google Docs and Sheets, etc.

Microsoft has been integrating AI technology into workplace emails, slideshows, and spreadsheets as part of its ongoing partnership with OpenAI, the company behind ChatGPT. Microsoft's Azure OpenAI service offers a powerful tool to enable these outcomes when leveraged with their data lake of more than two billion metadata and transactional elements.

Google has opened access to their Gemini Model based AI/chat search service. I have used various Google APIs for years in code I write. I have no favorites in the battle between tech giants, rather I am mostly interested in what they build that I can use in my own projects.

Hugging Face, creates LLMs and also hosts those developed by other companies, is working on open-source rivals to ChatGPT and will use AWS[6] for that as well. Cohere AI, Anthropic, Hugging Face, and Stability AI are some of the startups that are competing with OpenAI and Hugging Face APIs. Hugging Face is a great source of specialized models, that is, standard models that have been fine tuned for specific applications. I love that Hugging Face models can be run via their APIs and also self-hosted on our own servers and sometimes even on our laptops. Hugging Face is a fantastic resource and even though I use their models much less frequently in this book than OpenAI APIs, you should embrace the hosting and open source flexibility of Hugging Face. Starting in late 2023 I also stated heavily using the Ollama[7] platform for downloading and running models on my laptop. There is a chapter in this book on using Ollama. In this book I most freequently use OpenAI APIs because they are so widely used.

[6] https://iblnews.org/aws-partners-with-hugging-face-an-ai-startup-rival-to-chatgpt-working-on-open-source-models/
[7] https://ollama.com

Dear reader, I didn't write this book for developers working at established AI companies (although I hope such people find the material here useful). I wrote this book for small developers who want to scratch their own itch by writing tools that save them time. I also wrote this book hoping that it would help developers build capabilities into the programs they design and write that rival what the big tech companies are doing.

Getting Started With LangChain

LangChain[1] is a framework for building applications with large language models (LLMs) through chaining different components together. Some of the applications of LangChain are chatbots, generative question-answering, summarization, data-augmented generation and more. LangChain can save time in building chatbots and other systems by providing a standard interface for chains, agents and memory, as well as integrations with other tools and end-to-end examples. We refer to "chains" as sequences of calls (to an LLMs and a different program utilities, cloud services, etc.) that go beyond just one LLM API call. LangChain provides a standard interface for chains, many integrations with other tools, and end-to-end chains for common applications. Often you will find existing chains already written that meet the requirements for your applications.

For example, one can create a chain that takes user input, formats it using a PromptTemplate, and then passes the formatted response to a Large Language Model (LLM) for processing.

While LLMs are very general in nature which means that while they can perform many tasks effectively, they often can not directly provide specific answers to questions or tasks that require deep domain knowledge or expertise. LangChain provides a standard interface for agents, a library of agents to choose from, and examples of end-to-end agents.

LangChain Memory is the concept of persisting state between calls of a chain or agent. LangChain provides a standard interface for memory, a collection of memory implementations, and examples of chains/agents that use memory. LangChain provides a large collection of common utils to use in your application. Chains go beyond just a single LLM call, and are sequences of calls (whether to an LLM or a different utility). LangChain provides a standard interface for chains, lots of integrations with other tools, and end-to-end chains for common applications.

LangChain can be integrated with one or more model providers, data stores, APIs, etc. LangChain can be used for in-depth question-and-answer chat sessions, API interaction, or action-taking. LangChain can be integrated with Zapier's platform through a natural language API interface (we have an entire chapter dedicated to Zapier integrations).

Installing Necessary Packages

For the purposes of examples in this book, you might want to create a new Anaconda or other Python environment and install:

[1] https://api.python.langchain.com/en/stable/langchain_api_reference.html

```
pip install -U langchain langchain-openai faiss-cpu tiktoken
pip install -U langchain llama_index langchain-openai
pip install -U langchain-community langchainhub
pip install -U kor pydrive pandas rdflib
pip install -U google-search-results SPARQLWrapper
```

For the rest of this chapter we will use the subdirectory **langchain_getting_started** and in the next chapter use **llama-index_case_study** in the GitHub repository for this book.

Creating a New LangChain Project

Simple LangChain projects are often just a very short Python script file. As you read this book, when any example looks interesting or useful, I suggest copying the requirements.txt and Python source files to a new directory and making your own GitHub private repository to work in. Please make the examples in this book "your code," that is, freely reuse any code or ideas you find here.

Basic Usage and Examples

While I try to make the material in this book independent, something you can enjoy with no external references, you should also take advantage of the high quality Langchain Quickstart Guide documentation[2] and the individual detailed guides for prompts, chat, document loading, indexes, etc.

As we work through some examples please keep in mind what it is like to use the ChatGPT web application: you enter text and get responses. The way you prompt ChatGPT is obviously important if you want to get useful responses. In code examples we automate and formalize this manual process.

You need to choose a LLM to use. We will usually choose the GPT-4 API from OpenAI because it is general purpose but is more expensive that the older GPT-3.5 APIs. You will need to sign up[3] for an API key and set it as an environment variable:

```
export OPENAI_API_KEY="YOUR KEY GOES HERE"
```

Both the libraries **openai** and **langchain** will look for this environment variable and use it. We will look at a few simple examples in a Python REPL. We will start by just using OpenAI's text prediction API:

[2] https://python.langchain.com/docs/get_started/quickstart
[3] https://platform.openai.com/account/api-keys

```
$ python
>>> from langchain_openai import OpenAI
>>> llm = OpenAI(temperature=0.8)
>>> s = llm("John got into his new sports car, and he drove it")
>>> s
' to work\n\nJohn started up his new sports car and drove it to work. He had a huge \
smile on his face as he drove, excited to show off his new car to his colleagues. Th
e wind blowing through his hair, and the sun on his skin, he felt a sense of freedom
 and joy as he cruised along the road. He arrived at work in no time, feeling refres
hed and energized.'
>>> s = llm("John got into his new sports car, and he drove it")
>>> s
" around town\n\nJohn drove his new sports car around town, enjoying the feeling of \
the wind in his hair. He stopped to admire the view from a scenic lookout, and then
sped off to the mall to do some shopping. On the way home, he took a detour down a w
inding country road, admiring the scenery and enjoying the feel of the car's powerfu
l engine. By the time he arrived back home, he had a huge smile on his face."
```

Notice how when we ran the same input text prompt twice that we see different results. Setting the temperature in line 3 to a higher value increases the randomness.

Our next example is in the source file **directions_template.py** in the directory **langchain_getting_started** and uses the **PromptTemplate** class. A prompt template is a reproducible way to generate a prompt. It contains a text string ("the template"), that can take in a set of parameters from the end user and generate a prompt. The prompt template may contain language model instructions, few-shot examples to improve the model's response, or specific questions for the model to answer.

```
from langchain.prompts import PromptTemplate
from langchain_openai import OpenAI
llm = OpenAI(temperature=0.9)

def get_directions(thing_to_do):
    prompt = PromptTemplate(
        input_variables=["thing_to_do"],
        template="How do I {thing_to_do}?",
    )
    prompt_text = prompt.format(thing_to_do=thing_to_do)
    print(f"\n{prompt_text}:")
    return llm(prompt_text)

print(get_directions("get to the store"))
print(get_directions("hang a picture on the wall"))
```

You could just write Python string manipulation code to create a prompt but using the utility class **PromptTemplate** is more legible and works with any number of prompt input variables.

The output is:

```
$ python directions_template.py

How do I get to the store?:

To get to the store, you will need to use a mode of transportation such as walking, \
driving, biking, or taking public transportation. Depending on the location of the s
tore, you may need to look up directions or maps to determine the best route to take
.

How do I hang a picture on the wall?:

1. Find a stud in the wall, or use two or three wall anchors for heavier pictures.
2. Measure and mark the wall where the picture hanger will go.
3. Pre-drill the holes and place wall anchors if needed.
4. Hammer the picture hanger into the holes.
5. Hang the picture on the picture hanger.
```

The next example in the file **country_information.py** is derived from an example in the LangChain documentation. In this example we use **PromptTemplate** that contains the pattern we would like the LLM to use when returning a response.

```
from langchain.prompts import PromptTemplate
from langchain_openai import OpenAI
llm = OpenAI(temperature=0.9)

def get_country_information(country_name):
    print(f"\nProcessing {country_name}:")
    global prompt
    if "prompt" not in globals():
        print("Creating prompt...")
        prompt = PromptTemplate(
            input_variables=["country_name"],
            template = """
Predict the capital and population of a country.

Country: {country_name}
Capital:
Population:""",
```

```
        )
    prompt_text = prompt.format(country_name=country_name)
    print(prompt_text)
    return llm(prompt_text)

print(get_country_information("Canada"))
print(get_country_information("Germany"))
```

You can use the ChatGPT web interface to experiment with prompts and when you find a pattern that works well then write a Python script like the last example, but changing the data you supply in the **PromptTemplate** instance.

The output of the last example is:

```
 $ python country_information.py

Processing Canada:
Creating prompt...

Predict the capital and population of a country.

Country: Canada
Capital:
Population:


Capital: Ottawa
Population: 37,058,856 (as of July 2020)

Processing Germany:

Predict the capital and population of a country.

Country: Germany
Capital:
Population:


Capital: Berlin
Population: 83,02 million (est. 2019)
```

Creating Embeddings

We will reference the LangChain embeddings documentation[4]. We can use a Python REPL to see what text to vector space embeddings might look like:

```
$ python
Python 3.10.8 (main, Nov 24 2022, 08:08:27) [Clang 14.0.6 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> from langchain_openai import OpenAIEmbeddings
>>> embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
>>> text = "Mary has blond hair and John has brown hair. Mary lives in town and John\
 lives in the country."
>>> doc_embeddings = embeddings.embed_documents([text])
>>> doc_embeddings
[[0.007727328687906265, 0.0009025644976645708, -0.0033224383369088173, -0.0179449208\
0807686, -0.017969949170947075, 0.028506645932793617, -0.013414892368018627, 0.00466
76816418766975, -0.0024965214543044567, -0.02662956342101097,
...]]
>>> query_embedding = embeddings.embed_query("Does John live in the city?")
>>> query_embedding
[0.028048301115632057, 0.011499025858938694, -0.00944007933139801, -0.02080961130559\
4444, -0.023904507979750633, 0.018750663846731186, -0.01626438833773136, 0.018129095
435142517,
...]
>>>
```

Notice that the **doc_embeddings** is a list where each list element is the embeddings for one input text document. The **query_embedding** is a single embedding. Please read the above linked embedding documentation.

We will use vector stores to store calculated embeddings for future use. In the next chapter we will see a document database search example using LangChain and Llama-Index.

Using LangChain Vector Stores to Query Documents: a Simple RAG Application

We will reference the LangChain Vector Stores documentation[5].

We use a utility in file **read_text_files.py** to convert text files in a directory to a list of strings:

[4] https://python.langchain.com/en/latest/reference/modules/embeddings.html
[5] https://python.langchain.com/en/latest/reference/modules/vectorstore.html

```python
import os

# Generated by Perplexity

def read_text_files(directory_path):
    """
    Reads all .txt files in the specified directory and returns their contents as a \
list of strings.

    :param directory_path: The path to the directory containing .txt files
    :return: A list of strings where each string is the content of a .txt file
    """
    txt_contents = []
    # Check if the directory exists
    if not os.path.isdir(directory_path):
        print(f"The directory {directory_path} does not exist.")
        return txt_contents

    # Iterate over all files in the directory
    for filename in os.listdir(directory_path):
        # Check for .txt extension
        if filename.endswith(".txt"):
            # Construct full file path
            file_path = os.path.join(directory_path, filename)
            # Open and read the contents of the file
            try:
                with open(file_path, 'r') as file:
                    txt_contents.append(file.read())
            except IOError as e:
                print(f"Failed to read file {filename}: {e}")

    return txt_contents
```

The example script is **doc_search.py**:

```
from langchain_community.vectorstores import FAISS
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.document_loaders import DirectoryLoader

model = ChatOpenAI()

from read_text_files import read_text_files

vectorstore = FAISS.from_texts(read_text_files("../data/"), embedding=OpenAIEmbeddin\
gs())

retriever = vectorstore.as_retriever()

template = """Answer the question based only on the following context:
{context}

Question: {question}
"""
prompt = ChatPromptTemplate.from_template(template)

chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)

print(chain.invoke("who tried to define what Chemistry is?"))

print(chain.invoke("What kinds of equipment are in a chemistry laboratory?"))
print(chain.invoke("What is Austrian School of Economics?"))
print(chain.invoke("Why do people engage in sports?"))
print(chain.invoke("What is the effect of body chemistry on exercise?"))
```

The output is:

```
$ python doc_search.py
Georg Ernst Stahl, Jean-Baptiste Dumas, Linus Pauling, and Professor Raymond Chang t\
ried to define what Chemistry is.
Various forms of laboratory glassware are typically used in a chemistry laboratory. \
However, it is mentioned that a great deal of experimental and applied/industrial ch
emistry can also be done without the use of glassware.
The Austrian School of Economics is a school of economic thought that emphasizes the\
 spontaneous organizing power of the price mechanism, advocates a laissez-faire appr
oach to the economy, and holds that commercial transactions should be subject to min
imal government intervention.
People engage in sports for physical athleticism, physical dexterity, and for leisur\
e activities that they find amusing or entertaining.
Body chemistry can affect exercise in terms of how energy is transferred from one su\
bstance to another, with heat being transferred more easily than other forms of ener
gy. Additionally, the body's ability to learn to fight specific infections after exp
osure to germs, known as adaptive immunity, can impact exercise performance and reco
very.
```

Example Using LangChain Integrations: Using Server APIs for Google Search

NOTE: This example is deprecated.

The example shown here is in the directory **from_langchain_docs** in the source file **search_simple.py**. The relevant LangChain Integrations documentation page is https://python.langchain.com/docs/integrations/tools/google_serper.

```
# make sure SERPER_API_KEY is set in your environment

from langchain_community.utilities import GoogleSerperAPIWrapper
search_helper = GoogleSerperAPIWrapper()

def search(query):
    return search_helper.run(query)

print(search("What is the capital of Arizona?"))
#print(search("Sedona Arizona?"))
```

You will need a Server API key form https://serper.dev. Currently you can get a free key for 2500 API calls. After that the paid tier currently starts at $50 for 50K API calls and these credits must be used within a 6 month period.

OpenAI Model GPT-4o Example

Here we use the new OpenAI model GPT-4o released May 13, 2024. The example file **from_langchain_docs/gpt_4o_test.py** uses the latest (May 2024) LangChain APIs:

```
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o")

messages = [
    SystemMessage(content="You're a helpful assistant"),
    HumanMessage(content="What is the purpose of model regularization? Be concise."),
]

results = llm.invoke(messages)
print(results.content)
print("\n")
print(results)
```

The output is:

```
$ python gpt_4o_test.py
The purpose of model regularization is to prevent overfitting by adding a penalty fo\
r larger coefficients in the model, thereby improving its generalization to new, uns
een data.

content='The purpose of model regularization is to prevent overfitting by adding a p\
enalty for larger coefficients in the model, thereby improving its generalization to
 new, unseen data.' response_metadata={'token_usage': {'completion_tokens': 34, 'pro
mpt_tokens': 27, 'total_tokens': 61}, 'model_name': 'gpt-4o', 'system_fingerprint':
'fp_729ea513f7', 'finish_reason': 'stop', 'logprobs': None} id='run-f2fa3e57-eeff-44
96-b6e4-d06af8f230d0-0'
```

This new (as of May 2024) model is half the cost and much lower API call latency than the previous GPT-4 API.

LangChain Overview Wrap Up

We will continue using LangChain for the rest of this book as well as the LlamaIndex library that we introduce in the next chapter.

I cover just the subset of LangChain that I use in my own projects in this book. I urge you to read the LangChain documentation and to explore public LangChain chains that users have written on Langchain-hub[6].

[6] https://github.com/hwchase17/langchain-hub

Overview of LlamaIndex

The popular LlamaIndex project used to be called GPT-Index but has been generalized to work with many LLM models like GPT-4, Hugging Face, Anthropic, local models run using Ollama, and many other models.

LlamaIndex is a project that provides a central interface to connect your language models with external data. It was created by Jerry Liu and his team in the fall of 2022. It consists of a set of data structures designed to make it easier to use large external knowledge bases with language models. Some of its uses are:

- Querying structured data such as tables or databases using natural language
- Retrieving relevant facts or information from large text corpora
- Enhancing language models with domain-specific knowledge

LlamaIndex supports a variety of document types, including:

- Text documents are the most common type of document. They can be stored in a variety of formats, such as .txt, .doc, and .pdf.
- XML documents are a type of text document that is used to store data in a structured format.
- JSON documents are a type of text document that is used to store data in a lightweight format.
- HTML documents are a type of text document that is used to create web pages.
- PDF documents are a type of text document that is used to store documents in a fixed format.

LlamaIndex can also index data that is stored in a variety of databases, including:

- SQL databases such as MySQL, PostgreSQL, and Oracle. NoSQL databases such as MongoDB, Cassandra, and CouchDB.
- Solr is a popular open-source search engine that provides high performance and scalability.
- Elasticsearch is another popular open-source search engine that offers a variety of features, including full-text search, geospatial search, and machine learning.
- Apache Cassandra is a NoSQL database that can be used to store large amounts of data.
- MongoDB is another NoSQL database that is easy to use and scale.
- PostgreSQL is a relational database that is widely used in enterprise applications.

LlamaIndex is a flexible framework that can be used to index a variety of document types and data sources.

Compared to LangChain, LlamaIndex presents a focused advantage in the realm of indexing and retrieval tasks, making it a highly efficient choice for applications that prioritize these functions. Its design is tailored specifically for the efficient ingestion, structuring, and accessing of private or domain-specific data, which is crucial for applications that rely heavily on the quick retrieval of accurate and relevant information from large datasets. The streamlined interface of LlamaIndex simplifies the process of connecting custom data sources to large language models (LLMs), thereby reducing the complexity and development time for search-centric applications. This focus on indexing and retrieval, as highlighted in the search results, leads to increased speed and accuracy in search and summarization tasks, setting LlamaIndex apart as the go-to framework for developers building intelligent search tools.

Another significant advantage of LlamaIndex is its integration capabilities with a wide array of tools and services, which enhances the functionality and versatility of LLM-powered applications. The framework's ability to merge with vector stores like Pinecone and Milvus facilitates efficient document search and retrieval. Additionally, its compatibility with tracing tools such as Graphsignal offers insights into LLM-powered application operations, while integration with application frameworks like LangChain and Streamlit enables easier building and deployment. These integrations extend to data loaders, agent tools, and observability tools, thus enhancing the capabilities of data agents and offering various structured output formats to facilitate the consumption of application results. This extensive integration ecosystem empowers developers to create powerful, versatile applications with minimal effort.

Lastly, LlamaIndex's specialized focus on indexing and retrieval is complemented by its simplicity and ease of use, making it an attractive option for developers seeking to build efficient and straightforward search experiences. The framework's optimization for these specific tasks, in comparison to more general-purpose frameworks like LangChain, results in a tool that is not only more efficient for search and retrieval applications but also easier to learn and implement. This simplicity is particularly beneficial for projects with tight deadlines or for developers new to working with LLMs, as it allows for the quick deployment of high-performance applications without the need for extensive customization or complex setup processes.

We will look at a short example derived from the LlamaIndex documentation.

Using LlamaIndex for Question Answering from a Web Site

In this example we use the **trafilatura** and **html2text** libraries to get text from a web page that we will index and search. The class **TrafilaturaWebReader** does the work of creating local documents from a list of web page URIs and the index class **VectorStoreIndex** builds a local index for use with OpenAI API calls to implement search.

```
pip install -U trafilatura html2text
```

The following listing shows the file **web_page_QA.py**:

```
# Derived from examples in llama_index documentation

# pip install llama-index html2text trafilatura

from pprint import pprint
from llama_index.core import Document
import trafilatura

from llama_index.core import VectorStoreIndex

def query_website(url, *questions):
    downloaded = trafilatura.fetch_url(url)
    text = trafilatura.extract(downloaded)
    #print(text)
    list_of_documents = [Document(text=text)]
    index = VectorStoreIndex.from_documents(list_of_documents)   #.from_texts([text])
    engine = index.as_query_engine()
    for question in questions:
        print(f"\n== QUESTION: {question}\n")
        response = engine.query(question)
        print(f"== RESPONSE: {response}")

if __name__ == "__main__":
  url = "https://markwatson.com"
  query_website(url, "What instruments does Mark play?",
                     "How many books has Mark written?")
```

This example is not efficient because we create a new index for each web page we want to search. That said, this example (that was derived from an example in the LlamaIndex documentation) implements a pattern that you can use, for example, to build a reusable index of your company's web site and build an end-user web search app.

The output for these three test questions in the last code example is:

```
 $ python web_page_QA.py

== QUESTION: What instruments does Mark play?

== RESPONSE: Mark plays the guitar, didgeridoo, and American Indian flute.

== QUESTION: How many books has Mark written?

== RESPONSE: Mark has written 9 books.
```

Note that the answer to the second question is strictly incorrect since it counted the books mentioned in the text. It did this correctly. However, the Trafilatura library skipped the text in the header block of my web site that said I have written over 20 books. This inaccuracy if from my use of the Trafilatura library.

LlamaIndex Case Study Wrap Up

LlamaIndex is a set of data structures and library code designed to make it easier to use large external knowledge bases such as Wikipedia. LlamaIndex creates a vectorized index from your document data, making it highly efficient to query. It then uses this index to identify the most relevant sections of the document based on the query.

LlamaIndex is useful because it provides a central interface to connect your LLM's with external data and offers data connectors to your existing data sources and data formats (API's, PDF's, docs, SQL, etc.). It provides a simple, flexible interface between your external data and LLMs.

Some projects that use LlamaIndex include building personal assistants with LlamaIndex and GPT-4, using LlamaIndex for document retrieval, and combining answers across documents.

Extraction of Facts and Relationships from Text Data

Traditional methods for extracting email addresses, names, addresses, etc. from text included the use of hand-crafted regular expressions and custom software. LLMs are text processing engines with knowledge of grammar, sentence structure, and some real world embedded knowledge. Using LLMs can reduce the development time of information extraction systems.

Key Capabilities of LLMs for Fact and Relationship Extraction

- Named Entity Recognition (NER): LLMs excel at identifying and classifying named entities within text. This includes pinpointing people, organizations, locations, dates, quantities, etc. NER forms the basis of any fact extraction process, as entities are the core elements around which facts are organized.
- Relationship Extraction (RE): LLMs are adept at understanding the grammatical structure of sentences and the contextual meaning of words. This enables them to identify relationships between the entities they've identified, such as: Employment relationships ("Jane Smith works for Microsoft") Ownership ("Apple acquired Beats Electronics") and Location-based relationships ("The Louvre Museum is located in Paris")
- Semantic Understanding: LLMs possess a deep understanding of language semantics. This allows them to go beyond simple keyword matching and grasp the nuances and implicit meanings within text, leading to more accurate fact extraction.
- Knowledge Base Augmentation: Pre-trained LLMs draw on their vast knowledge bases (from being trained on massive text datasets) to fill in gaps when text is incomplete and support the disambiguation of entities or relationships.

Techniques and Approaches

- Fine-tuned Question Answering: LLMs can be fine-tuned to directly answer factual questions posed based on a text. For example, given a news article and the question, "When did the event occur?", the LLM can pin down the relevant date within the text.
- Knowledge Graph Construction: LLMs play a crucial role in automatically constructing knowledge graphs. These graphs are structured representations of facts and relationships extracted from text. LLMs identify the entities, relationships, and help enrich the graphs with relevant attributes.

- Zero-shot or Few-shot Learning: Advanced LLMs can extract certain facts and relationships with minimal or no additional training examples. This is especially valuable in scenarios where manually labelled data is scarce or time-consuming to create.

Benefits

- Accuracy: LLMs often surpass traditional rule-based systems in accuracy, particularly when working with complex or varied text formats.
- Scalability: LLMs can process vast amounts of text data to efficiently extract facts and relationships, enabling the analysis of large-scale datasets.
- Time-saving: The ability of LLMs to adapt and learn reduces the need for extensive manual rule creation or feature engineering, leading to faster development of fact extraction systems.

Applications

- Financial Analysis: Identifying key facts and relationships within financial reports and news articles to support investment decisions.
- Legal Research: Extracting relevant clauses, case law, and legal relationships from complex legal documents.
- Scientific Literature Analysis: Building databases of scientific findings and discoveries by extracting relationships and networks from research papers.
- Customer Support: Analyzing customer feedback and queries to understand product issues, sentiment, and commonly reported problems.

Example Prompts for Getting Information About a Person from Text and Generating JSON

Before using LLMs directly in application code I like to experiment with prompts. Here we will use a two-shot approach of providing as context two examples of text and the extracted JSON data, followed by text we want to process. Consider the following that I ran on my old M1 8G MacBook:

```
Given the two examples below, extract the names, addresses, and email addresses of i\
ndividuals mentioned later as Process Text. Format the extracted information in JSON
, with keys for "name", "address", and "email". If any information is missing, use "
null" for that field.

Example 1:
Text: "John Doe lives at 1234 Maple Street, Springfield. His email is johndoe@exampl\
e.com."
Output:
{
  "name": "John Doe",
  "address": "1234 Maple Street, Springfield",
  "email": "johndoe@example.com"
}

Example 2:
Text: "Jane Smith has recently moved to 5678 Oak Avenue, Anytown. She hasn't updated\
 her email yet."
Output:
{
  "name": "Jane Smith",
  "address": "5678 Oak Avenue, Anytown",
  "email": null
}

Process Text: "Mark Johnson enjoys living in Berkeley California at 102 Dunston Stre\
et and use mjess@foobar.com for contacting him."
Output:
```

This prompt is in the file **prompt_examples/two-shot-2.txt**.

The output can be overly verbose:

```
$ ollama run llama3:instruct < two-shot-2.txt
Here is the extracted information in JSON format:

{
"name": "Mark Johnson",
"address": "102 Dunston Street, Berkeley California",
"email": "mjess@foobar.com"
}

Note that I used the address format from Example 1, which combines the street
```

```
address with the city and state. If you want to separate these fields into different
keys (e.g., "street", "city", "state"), let me know!
```

While the comments the llama3-8b-instruct model makes are interesting, let's modify the prompt to ask for concise output that only includes the generated JSON:

```
Given the two examples below, extract the names, addresses, and email addresses of i\
ndividuals mentioned later as Process Text. Format the extracted information in JSON
, with keys for "name", "address", and "email". If any information is missing, use "
null" for that field. Be concise in your output by providing only the output JSON.
```

The rest of the prompt is unchanged, now the output is:

```
$ ollama run llama3:instruct < two-shot-2.txt
{
  "name": "Mark Johnson",
  "address": "102 Dunston Street, Berkeley California",
  "email": "mjess@foobar.com"
}
```

Example Code

To use this example we would use the same prompt except we would make the **Process Text** a variable that is replaced before processing by an LLM. We copy the file **two-shot-2.txt** to **two-shot-2-var.txt** and change the second to the last line in the file:

```
Process Text: "{input_text}"
```

Now let's wrap these ideas up in a short Python example in the file **extraction/person_data.py**:

```
import openai
from openai import OpenAI
import os

openai.api_key = os.getenv("OPENAI_API_KEY")
client = OpenAI()

# Read the prompt from a text file
with open('prompt.txt', 'r') as file:
    prompt_template = file.read()
```

```
11
12 # Substitute a string variable into the prompt
13 input_text = "Mark Johnson enjoys living in Berkeley California at 102 Dunston Stree\
14 t and use mjess@foobar.com for contacting him."
15 prompt = prompt_template.replace("input_text", input_text)
16
17 # Use the OpenAI completion API to generate a response with GPT-4
18 completion = client.chat.completions.create(
19     model="gpt-4",
20     messages=[
21         {
22             "role": "user",
23             "content": prompt,
24         },
25     ],
26 )
27
28 print(completion.choices[0].message.content)
```

The output looks like:

```
1 $ python person_data.py
2 {
3   "name": "Mark Johnson",
4   "address": "102 Dunston Street, Berkeley California",
5   "email": "mjess@foobar.com"
6 }
```

For reference, the complete **completion** object looks like this:

```
1 ChatCompletion(id='chatcmpl-9LBZao4hFMmw7VrYcRbQIR2EGzvCj', choices=[Choice(finish_r\
2 eason='stop', index=0, logprobs=None, message=ChatCompletionMessage(content='{\n  "n
3 ame": "Mark Johnson",\n  "address": "102 Dunston Street, Berkeley California",\n  "e
4 mail": "mjess@foobar.com"\n}', role='assistant', function_call=None, tool_calls=None
5 ))], created=1714836402, model='gpt-4-0613', object='chat.completion', system_finger
6 print=None, usage=CompletionUsage(completion_tokens=34, prompt_tokens=223, total_tok
7 ens=257))
```

Using LLMs to Summarize Text

LLMs bring a new level of ability to text summarization tasks. With their ability to process massive amounts of information and "understand" natural language, they're able to capture the essence of lengthy documents and distill them into concise summaries. Two main types of summarization dominate with LLMs: extractive and abstractive. Extractive summarization pinpoints the most important sentences within the original text, while abstractive summarization requires the LLM to paraphrase or generate new text to represent the core ideas. If you are interested in extractive summarization there is a chapter on this topic in my Common Lisp AI book[1] (link to read online).

LLMs excel in text summarization for several reasons. Their deep understanding of language semantics allows them to identify key themes, even when wording varies across a document. Additionally, they have an ability for maintaining logical consistency within summaries, ensuring that the condensed version makes sense as a cohesive unit. Modern LLMs are also trained on massive datasets encompassing diverse writing styles, helping them adapt to different sources and generate summaries tailored to specific audiences.

The applications of LLM-powered text summarization are vast. They can help researchers digest lengthy scientific reports quickly, allow businesses to analyze customer feedback efficiently, or provide concise news briefs for busy individuals. LLM-based summarization also has the potential to improve accessibility, creating summaries for those with reading difficulties or summarizing complex information into simpler language. As LLM technology continues to advance, we can expect even more innovative and accurate summarization tools in the future.

Example Prompt

In this example, the prompt is simply:

```
Summarize the following text: "{input_text}"
Output:
```

Code Example

The example in file **summarization/summarization_example.py** reads a prompt file and substitutes the text from the test file **../data/economics.txt**:

[1] https://leanpub.com/lovinglisp/read

```python
import openai
from openai import OpenAI
import os

openai.api_key = os.getenv("OPENAI_API_KEY")
client = OpenAI()

# Read the prompt from a text file
with open('prompt.txt', 'r') as file:
    prompt_template = file.read()

# Substitute a string variable into the prompt
with open('../data/economics.txt', 'r') as file:
    input_text = file.read()
prompt = prompt_template.replace("input_text", input_text)

# Use the OpenAI completion API to generate a response with GPT-4
completion = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {
            "role": "user",
            "content": prompt,
        },
    ],
)

print(completion.choices[0].message.content)
```

The length of the output summary is about 20% of the length of the original text.

LLM Techniques for Structured Data Conversion

Here we look at a simple example of converting CSV spreadsheet files to JSON but the idea of data conversion using LLMs is general purpose.

Using LLMs helps handle ambiguity. Traditional Symbolic AI methods often struggle with the nuance of human language. LLMs, with their understanding of context, can resolve ambiguity and provide more accurate extraction.

LLMs are also effective at handling complex or previously unseen formats (one shot). LLMs are trained on vast amounts of diverse text data, making them more adaptable to unexpected variations in data formats than rule-based approaches.

Using LLMs for application development can reduce manual effort by automating many parts of the conversion process that traditionally required significant human intervention and the creation of detailed extraction rules.

Example Prompt for Converting CSV Files to JSON

In the prompt we supply a few examples for converting between these two formats:

```
Given the example below, convert a CSV spreadsheet text file to a JSON text file:

Example:
CSV:
name,address, email
John Doe, 1234 Maple Street, Springfield,johndoe@example.com
"Jane Smith", "5678 Oak Avenue, Anytown", jane@smith764323.com
Output:
{
  "name": "John Doe",
  "address": "1234 Maple Street, Springfield",
  "email": "johndoe@example.com"
}
{
  "name": "Jane Smith",
  "address": "5678 Oak Avenue, Anytown",
  "email": null
```

```
}

Process Text: "{input_csv}"
Output:
```

Example Code for Converting CSV Files to JSON

The example in file **structured_data_conversion/person_data.py** reads the prompt template file and substitutes the CSV data from the test file **test.csv**. The modified prompt is passed to the OpenAI completion API:

```
import openai
from openai import OpenAI
import os

openai.api_key = os.getenv("OPENAI_API_KEY")
client = OpenAI()

# Read the prompt from a text file
with open('prompt.txt', 'r') as file:
    prompt_template = file.read()

# Substitute a string variable into the prompt
with open('test.csv', 'r') as file:
    input_csv = file.read()
prompt = prompt_template.replace("input_csv", input_csv)

# Use the OpenAI completion API to generate a response with GPT-4
completion = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {
            "role": "user",
            "content": prompt,
        },
    ],
)

print(completion.choices[0].message.content)
```

Here is the test CSV input file:

```
last_name,first_name,email
"Jackson",Michael,mj@musicgod.net
Jordan,Michael,"mike@retired.com"
Smith, John, john@acme41.com
```

Notice that this file is not consistent in quoting strings, hopefully making this a more general example of data you might see in the *wild*. The generated JSON looks like:

```
{
  "last_name": "Jackson",
  "first_name": "Michael",
  "email": "mj@musicgod.net"
}
{
  "last_name": "Jordan",
  "first_name": "Michael",
  "email": "mike@retired.com"
}
{
  "last_name": "Smith",
  "first_name": "John",
  "email": "john@acme41.com"
}
```

Retrieval Augmented Generation (RAG) Applications

Note: August 27, 2024: pinned library versions.

Retrieval Augmented Generation (RAG) Applications work by pre-processing a user's query to search for indexed document fragments that are semantically similar to the user's query. These fragments are concatenated together as context text that is attached to the user's query and then passed of to a LLM model. The LLM can preferentially use information in this context text as well as innate knowledge stored in the LLM to process user queries.

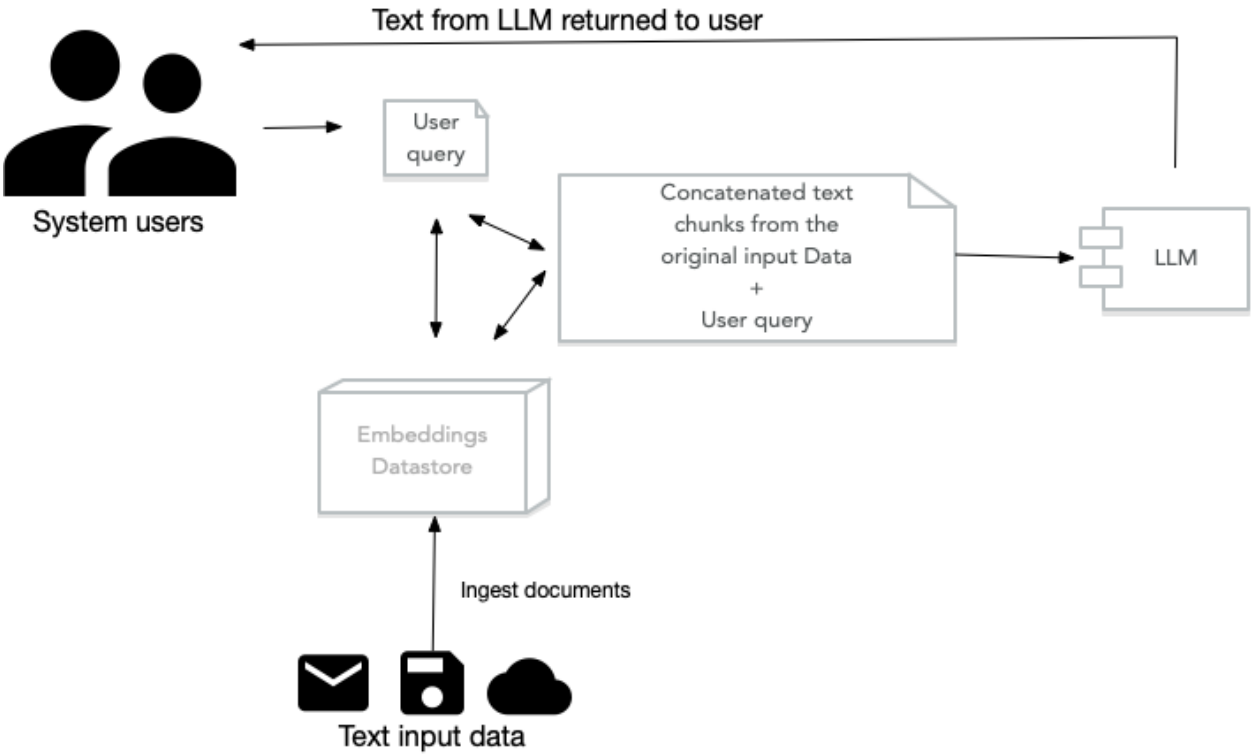


Figure 2. RAG System Overview

Simple RAG Example Using LlamaIndex

We will start with an example that only uses a vector store to find documents similar to the text in a query. Here is a listing of **rag/simple_llama_index_retrieve_docs.py**.

The code imports **VectorStoreIndex** and **Document** from the **llama_index.core** module. It then defines a list of strings, each describing aspects of LlamaIndex, and converts these strings into **Document** objects. These documents are then used to create an index using **VectorStoreIndex.from_documents(documents)**, which builds an index from the provided documents. This index is capable of understanding and storing the text data in a structured form that can be efficiently queried.

Following the index creation, the code initializes a query engine with **index.as_query_engine()**, which allows for querying the indexed data. The query "What is LlamaIndex?" is passed to the retrieve method of the query engine. This method processes the query against the indexed documents to find relevant information. The results are then printed. This demonstrates a basic use case of LlamaIndex for text retrieval where the system identifies and retrieves information directly related to the user's query from the indexed data. In practice we usually combine the use of vector stores with LLM chat models, as we do in later examples.

```
from llama_index.core import VectorStoreIndex
from llama_index.core import Document

text_list = ["LlamaIndex is a powerful tool for LLM applications.",
             "It helps in structuring and retrieving data efficiently."]
documents = [Document(text=t) for t in text_list]

index = VectorStoreIndex.from_documents(documents)
query_engine = index.as_query_engine()

retrieved_docs = query_engine.retrieve("What is LlamaIndex?")
print(retrieved_docs)
```

The output looks like:

```
 $ python simple_llama_index_retrieve_docs.py
[NodeWithScore(node=TextNode(id_='504482a2-e633-4d08-9bbe-363b42e67cde', embedding=N\
one, metadata={}, excluded_embed_metadata_keys=[], excluded_llm_metadata_keys=[], re
lationships={<NodeRelationship.SOURCE: '1'>: RelatedNodeInfo(node_id='876b1b31-116e-
4c31-a65d-1d9ae3bbe6a4', node_type=<ObjectType.DOCUMENT: '4'>, metadata={}, hash='7d
d73651a571de869137f2fc94f74d158ecbd7901ed81576ad7172de96394464')}, text='LlamaIndex
is a powerful tool for LLM applications.', start_char_idx=0, end_char_idx=51, text_t
emplate='{metadata_str}\n\n{content}', metadata_template='{key}: {value}', metadata_
```

```
seperator='\n'), score=0.9153632840140399), NodeWithScore(node=TextNode(id_='694d071
9-b6cc-4053-9c29-2cba0fc1f564', embedding=None, metadata={}, excluded_embed_metadata
_keys=[], excluded_llm_metadata_keys=[], relationships={<NodeRelationship.SOURCE: '1
'>: RelatedNodeInfo(node_id='b9411d72-f945-4553-8cb4-5b55754e9b56', node_type=<Objec
tType.DOCUMENT: '4'>, metadata={}, hash='2813e9c7d682d7dec242363505166e9696fbf7a4958
ec6e1c2ed8e47389f70c6')}, text='It helps in structuring and retrieving data efficien
tly.', start_char_idx=0, end_char_idx=56, text_template='{metadata_str}\n\n{content}
', metadata_template='{key}: {value}', metadata_seperator='\n'), score=0.74424191545
64265)]
```

I almost never use just a vector index by itself. The next example is in the file **simple_rag_llama_index.py**.

This code demonstrates the use of the LlamaIndex library to process, index, and query text data, specifically focusing on extracting information about sports from a dataset. The code imports necessary classes from the **llama_index** library. **VectorStoreIndex** is used for creating a searchable index of documents.

SimpleDirectoryReader reads documents from a specified directory. **OpenAIEmbedding** is used to convert text into numerical embeddings using OpenAI's models. **SentenceSplitter** and **TitleExtractor** are used for preprocessing the text by splitting it into sentences and extracting titles, respectively. The ingestion pipeline configuration object **IngestionPipeline** is configured with three transformations:

- SentenceSplitter breaks the text into smaller chunks or sentences with specified chunk_size and chunk_overlap. This helps in managing the granularity of text processing.
- TitleExtractor pulls out titles from the text, which can be useful for summarizing or categorizing documents.
- OpenAIEmbedding converts the processed text into vector embeddings. These embeddings represent the text in a high-dimensional space, capturing semantic meanings which are crucial for effective searching and querying.

We use **SimpleDirectoryReader** to convert the text files in a directory to a list or document objects:

```
documents = SimpleDirectoryReader("../data_small").load_data()
```

As in the last example, we create a vector store object fro the list of document objects:

```
index = VectorStoreIndex.from_documents(documents)
```

The index is built using the vector embeddings generated by OpenAIEmbedding, allowing for semantic search capabilities.

The statement

```
query_engine = index.as_query_engine()
```

sets up a query engine from the index. This engine can process queries to find relevant documents based on their semantic content.

Finally we are ready to make a test query:

```
response = query_engine.query("List a few sports")
```

The engine searches through the indexed documents and retrieves information that semantically matches the query about sports.

Here is a complete code listing for this example:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.core.node_parser import SentenceSplitter
from llama_index.core.extractors import TitleExtractor
from llama_index.core.ingestion import IngestionPipeline

pipeline = IngestionPipeline(
    transformations=[
        SentenceSplitter(chunk_size=25, chunk_overlap=0),
        TitleExtractor(),
        OpenAIEmbedding(),
    ]
)

documents = SimpleDirectoryReader("../data_small").load_data()
index = VectorStoreIndex.from_documents(documents)

query_engine = index.as_query_engine()
response = query_engine.query("List a few sports")

print(response)
```

The output looks like:

```
$ python simple_rag_llama_index.py
Football, basketball, tennis, swimming, and cycling.
```

RAG With Reranking Example

Reranking in the context of Retrieval-Augmented Generation (RAG) for LLMs refers to a process of adjusting the order of documents retrieved by an initial search query to improve the relevance and quality of the results before they are used for generating responses. This step is crucial because the initial retrieval might fetch a broad set of documents, not all of which are equally relevant to the user's query.

The primary function of reranking is to refine the selection of documents based on their actual relevance to the expanded query. This is typically achieved by employing more sophisticated models that can better understand the context and nuances of the query and the documents. For instance, cross-encoder models are commonly used in reranking due to their ability to process the query and document simultaneously, providing a more accurate evaluation of relevance.

Just as LLMs have the flexibility to handle a broad range of text topics, different programming languages, etc., reranking mechanisms have the flexibility to handle a wide range of source information as well as a wide range of query types.

For your RAG applications, you should notice a reduction of noise text and irrelevance in RAG system responses to user queries.

The code example is very similar to the last example but we now add a reranker as a query engine postprocessor:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.core.node_parser import SentenceSplitter
from llama_index.core.extractors import TitleExtractor
from llama_index.core.ingestion import IngestionPipeline
from llama_index.core.postprocessor import SentenceTransformerRerank

# Set up the ingestion pipeline with transformations
pipeline = IngestionPipeline(
    transformations=[
        SentenceSplitter(chunk_size=25, chunk_overlap=0),
        TitleExtractor(),
        OpenAIEmbedding(),
    ]
)

# Load documents using a directory reader
documents = SimpleDirectoryReader("../data_small").load_data()

# Create an index from the documents
```

```
index = VectorStoreIndex.from_documents(documents)

# Initialize the reranker with a specific model
reranker = SentenceTransformerRerank(
    model="cross-encoder/ms-marco-MiniLM-L-12-v2",  # Example model, adjust as needed
    top_n=3  # Adjust the number of top documents to rerank
)

# Set up the query engine with the reranker as a postprocessor
query_engine = index.as_query_engine(
    similarity_top_k=10,  # Set for how many results to retrieve before reranking
    node_postprocessors=[reranker]  # Add the reranker to the postprocessing steps
)

# Perform a query
response = query_engine.query("List a few sports")

# Print the response
print(response)
```

The output is similar to before:

```
List a few sports: basketball, soccer, tennis, swimming, and cycling.
```

Let's try a more complex query and instead of just using the document directory **../data_small** that only contains information about sports, we will now use the text documents in **../data** that cover more general topics. We make two code changes. First we use a different document directory:

```
documents = SimpleDirectoryReader("../data").load_data()
```

We also change the query:

```
response = query_engine.query("Compare sports with the study of health issues")
```

The output looks like:

```
Sports are activities based on physical athleticism or dexterity, often governed by \
rules to ensure fair competition. On the other hand, the study of health issues invo
lves analyzing the production, distribution, and consumption of goods and services r
elated to health and well-being. While sports focus on physical activities and exerc
ise for leisure or competition, the study of health issues delves into understanding
 and addressing various aspects of physical and mental well-being, including disease
s, treatments, and preventive measures.
```

RAG on CSV Spreadsheet Files

TBD

Using Google's Knowledge Graph APIs With LangChain

Google's Knowledge Graph (KG) is a knowledge base that Google uses to serve relevant information in an info-box beside its search results. It allows the user to see the answer in a glance, as an instant answer. The data is generated automatically from a variety of sources, covering places, people, businesses, and more. I worked at Google in 2013 on a project that used their KG for an internal project.

Google's public Knowledge Graph Search API lets you find entities in the Google Knowledge Graph. The API uses standard schema.org types and is compliant with the JSON-LD specification. It supports entity search and lookup.

You can use the Knowledge Graph Search API to build applications that make use of Google's Knowledge Graph. For example, you can use the API to build a search engine that returns results based on the entities in the Knowledge Graph.

In the next chapter we also use the public KGs DBPedia and Wikidata. One limitation of Google's KG APIs is that it is designed for entity (people, places, organizations, etc.) lookup. When using DBPedia and Wikidata it is possible to find a wider range of information using the SPARQL query language, such as relationships between entities. You can use the Google KG APIs to find some entity relationships, e.g., all the movies directed by a particular director, or all the books written by a particular author. You can also use the API to find information like all the people who have worked on a particular movie, or all the actors who have appeared in a particular TV show.

Setting Up To Access Google Knowledge Graph APIs

To get an API key for Google's Knowledge Graph Search API, you need to go to the Google API Console, enable the Google Knowledge Graph Search API, and create an API key to use in your project. You can then use this API key to make requests to the Knowledge Graph Search API.

To create your application's API key, follow these steps:

- Go to the API Console.
- From the projects list, select a project or create a new one.
- If the APIs & services page isn't already open, open the left side menu and select APIs & services.
- On the left, choose Credentials.
- Click Create credentials and then select API key.

You can then use this API key to make requests to the Knowledge Graph Search APIs.

When I use Google's APIs I set the access key in ~/**.google_api_key** and read in the key using:

```
api_key=open(str(Path.home())+"/.google_api_key").read()
```

You can also use environment variables to store access keys. Here is a code snippet for making an API call to get information about me:

```
import json
from urllib.parse import urlencode
from urllib.request import urlopen
from pathlib import Path
from pprint import pprint

api_key =
    open(str(Path.home()) + "/.google_api_key").read()
query = "Mark Louis Watson"
service_url =
    "https://kgsearch.googleapis.com/v1/entities:search"
params = {
    "query": query,
    "limit": 10,
    "indent": True,
    "key": api_key,
}
url = service_url + "?" + urlencode(params)
response = json.loads(urlopen(url).read())
pprint(response)
```

The JSON-LD output would look like:

```
{'@context': {'@vocab': 'http://schema.org/',
              'EntitySearchResult':
              'goog:EntitySearchResult',
              'detailedDescription':
              'goog:detailedDescription',
              'goog': 'http://schema.googleapis.com/',
              'kg': 'http://g.co/kg',
              'resultScore': 'goog:resultScore'},
 '@type': 'ItemList',
 'itemListElement': [{'@type': 'EntitySearchResult',
                      'result': {'@id': 'kg:/m/0b6_g82',
```

```
                                '@type': ['Thing',
                                          'Person'],
                                'description': 'Author',
                                'name':
                                'Mark Louis Watson',
                                'url':
                                'http://markwatson.com'},
                    'resultScore': 43}]}
```

In order to not repeat the code for getting entity information from the Google KG, I wrote a utility **Google_KG_helper.py** that encapsulates the previous code and generalizes it into a mini-library.

```
"""Client for calling Knowledge Graph Search API."""

import json
from urllib.parse import urlencode
from urllib.request import urlopen
from pathlib import Path
from pprint import pprint

api_key =
    open(str(Path.home()) + "/.google_api_key").read()

# use Google search API to get information
# about a named entity:

def get_entity_info(entity_name):
    service_url =
      "https://kgsearch.googleapis.com/v1/entities:search"
    params = {
        "query": entity_name,
        "limit": 1,
        "indent": True,
        "key": api_key,
    }
    url = service_url + "?" + urlencode(params)
    response = json.loads(urlopen(url).read())
    return response

def tree_traverse(a_dict):
    ret = []
    def recur(dict_2, a_list):
        if isinstance(dict_2, dict):
```

```
            for key, value in dict_2.items():
                if key in ['name', 'description',
                           'articleBody']:
                    a_list += [value]
                recur(value, a_list)
        if isinstance(dict_2, list):
            for x in dict_2:
                recur(x, a_list)
    recur(a_dict, ret)
    return ret


def get_context_text(entity_name):
    json_data = get_entity_info(entity_name)
    return ' '.join(tree_traverse(json_data))

if __name__ == "__main__":
    get_context_text("Bill Clinton")
```

The main test script is in the file **Google_Knowledge_Graph_Search.py**:

```
"""Example of Python client calling the
   Knowledge Graph Search API."""

from llama_index.core.schema import Document
from llama_index.core import VectorStoreIndex
import Google_KG_helper

def kg_search(entity_name, *questions):
    ret = ""
    context_text = Google_KG_helper.get_context_text(entity_name)
    print(f"Context text: {context_text}")
    doc = Document(text=context_text)
    index = VectorStoreIndex.from_documents([doc])
    for question in questions:
        response = index.as_query_engine().query(question)
        ret += f"QUESTION:  {question}\nRESPONSE: {response}\n"
    return ret

if __name__ == "__main__":
    s = kg_search("Bill Clinton",
                  "When was Bill president?")
    print(s)
```

The example output is:

```
$ python Google_Knowledge_Graph_Search.py
Context text: William Jefferson Clinton is an American politician who served as the \
42nd president of the United States from 1993 to 2001. A member of the Democratic Pa
rty, he previously served as Governor of Arkansas from 1979 to 1981 and again from 1
983 to 1992.  42nd U.S. President Bill Clinton
QUESTION:  When was Bill president?
RESPONSE: Bill Clinton was president from 1993 to 2001.
```

Accessing Knowledge Graphs from Google, DBPedia, and Wikidata allows you to integrate real world facts and knowledge with your applications. While I mostly work in the field of deep learning I frequently also use Knowledge Graphs in my work and in my personal research. I think that you, dear reader, might find accessing highly structured data in KGs to be more reliable and in many cases simpler than using web scraping.

Using DBPedia and WikiData as Knowledge Sources

Both DBPedia[1] and Wikidata[2] are public Knowledge Graphs (KGs) that store data as Resource Description Framework (RDF)[3] and are accessed through the SPARQL Query Language for RDF[4]. The examples for this project are in the GitHub repository for this book[5] in the directory **kg_search**.

I am not going to spend much time here discussing RDF and SPARQL. Instead I ask you to read online the introductory chapter **Linked Data, the Semantic Web, and Knowledge Graphs** in my book A Lisp Programmer Living in Python-Land: The Hy Programming Language[6].

As we saw in the last chapter, a Knowledge Graph (that I often abbreviate as KG) is a graph database using a schema to define types (both objects and relationships between objects) and properties that link property values to objects. The term "Knowledge Graph" is both a general term and also sometimes refers to the specific Knowledge Graph used at Google which I worked with while working there in 2013. Here, we use KG to reference the general technology of storing knowledge in graph databases.

DBPedia and Wikidata are similar, with some important differences. Here is a summary of some similarities and differences between DBPedia and Wikidata:

- Both projects aim to provide structured data from Wikipedia in various formats and languages. Wikidata also has data from other sources so it contains more data and more languages.
- Both projects use RDF as a common data model and SPARQL as a query language.
- DBPedia extracts data from the infoboxes in Wikipedia articles, while Wikidata collects data entered through its interfaces by both users and automated bots.
- Wikidata requires sources for its data, while DBPedia does not.
- DBpedia is more popular in the Semantic Web and Linked Open Data communities, while Wikidata is more integrated with Wikimedia projects.

To the last point: I personally prefer DBPedia when experimenting with the semantic web and linked data, mostly because DBPedia URIs are human readable while Wikidata URIs are abstract. The following URIs represent the town I live in, Sedona Arizona:

- DBPedia: https://dbpedia.org/page/Sedona,_Arizona

[1] https://www.dbpedia.org
[2] https://www.wikidata.org/wiki/Wikidata:Main_Page
[3] https://www.w3.org/RDF/
[4] https://www.w3.org/TR/rdf-sparql-query/
[5] https://github.com/mark-watson/langchain-book-examples
[6] https://leanpub.com/hy-lisp-python/read

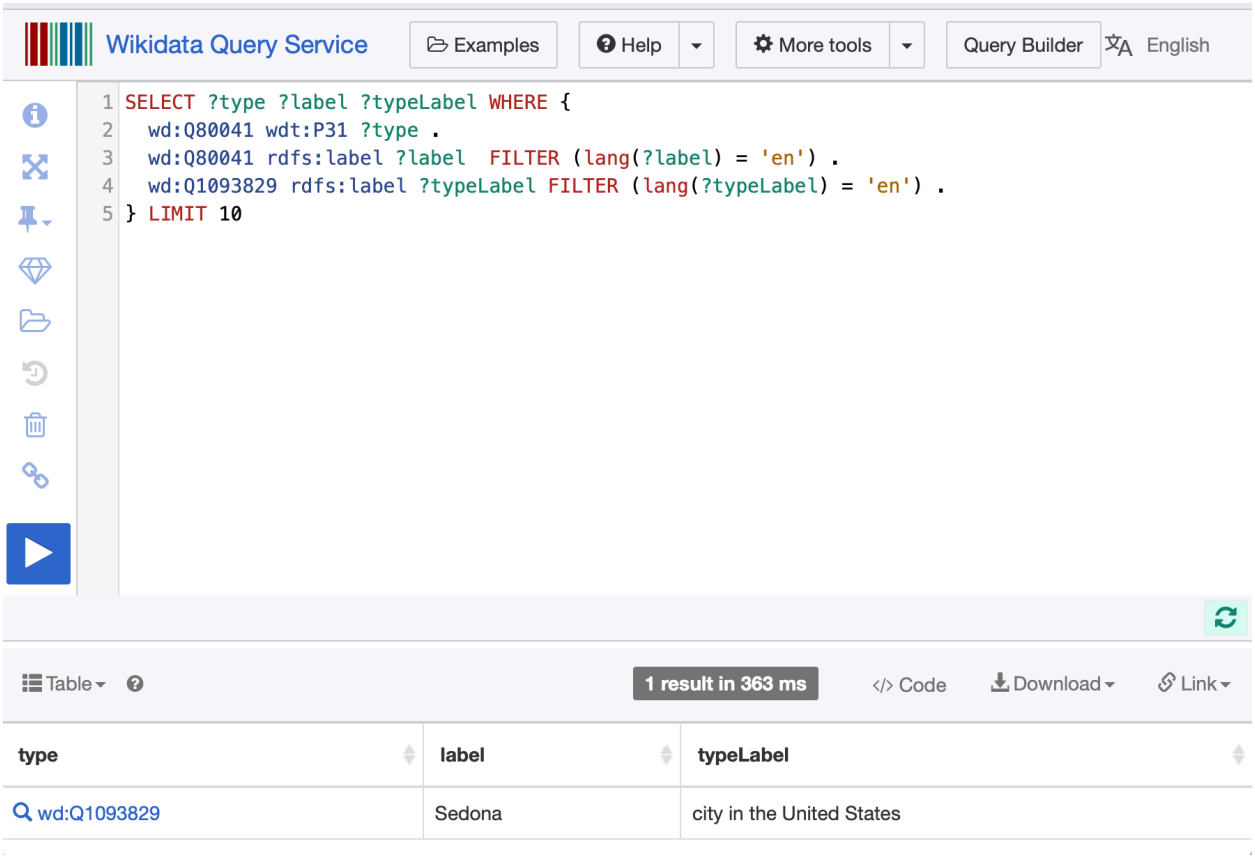
- Wikidata: https://www.wikidata.org/wiki/Q80041

In RDF we enclose URIs in angle brackets like **<https://www.wikidata.org/wiki/Q80041>**.

If you read the chapter on RDF and SPARQL in my book link that I mentioned previously, then you know that RDF data is represented by triples where each part is named:

- subject
- property
- object

We will look at two similar examples in this chapter, one using DBPedia and one using Wikidata. Both services have SPARQL endpoint web applications that you will want to use for exploring both KGs. We will look at the DBPedia web interface later. Here is the Wikidata web interface:

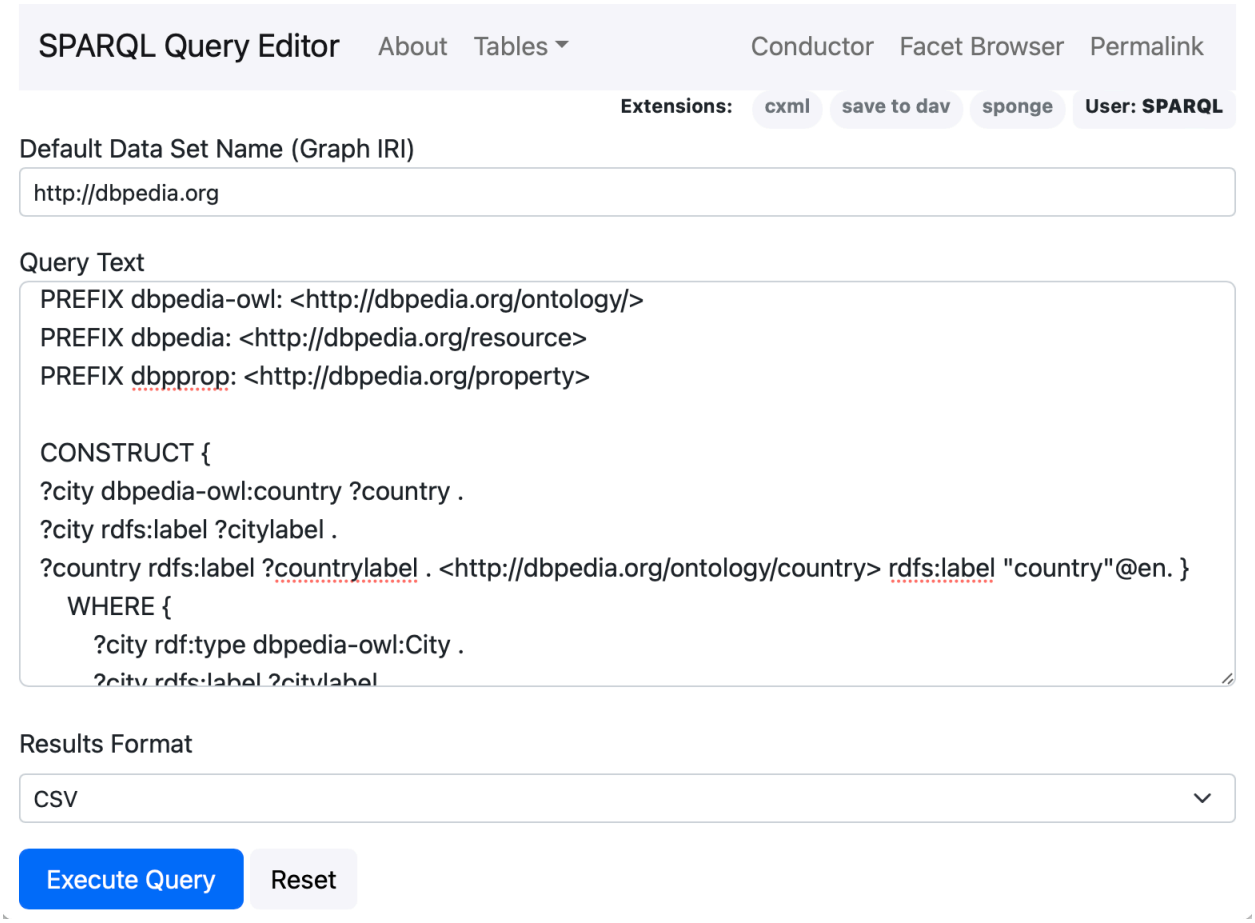


In this SPARQL query the prefix **wd:** stands for Wikidata data while the prefix **wdt:** stands for Wikidata type (or property). The prefix **rdfs:** stands for RDF Schema.

Using DBPedia as a Data Source

DBpedia is a community-driven project that extracts structured content from Wikipedia and makes it available on the web as a Knowledge Graph (KG). The KG is a valuable resource for researchers and developers who need to access structured data from Wikipedia. With the use of SPARQL queries to DBpedia as a data source we can write a variety applications, including natural language processing, machine learning, and data analytics. We demonstrate the effectiveness of DBpedia as a data source by presenting several examples that illustrate its use in real-world applications. In my experience, DBpedia is a valuable resource for researchers and developers who need to access structured data from Wikipedia.

In general you will start projects using DBPedia by exploring available data using the web app https://dbpedia.org/sparql that can be seen in this screen shot:



The following listing of file **dbpedia_generate_rdf_as_nt.py** shows Python code for making a SPARQL query to DBPedia and saving the results as RDF triples in NT format in a local text file:

```
from SPARQLWrapper import SPARQLWrapper
from rdflib import Graph

sparql = SPARQLWrapper("http://dbpedia.org/sparql")
sparql.setQuery("""
    PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>
    PREFIX dbpedia: <http://dbpedia.org/resource>
    PREFIX dbpprop: <http://dbpedia.org/property>

    CONSTRUCT {
        ?city dbpedia-owl:country ?country .
        ?city rdfs:label ?citylabel .
        ?country rdfs:label ?countrylabel .
        <http://dbpedia.org/ontology/country> rdfs:label "country"@en .
    }
    WHERE {
        ?city rdf:type dbpedia-owl:City .
        ?city rdfs:label ?citylabel .
        ?city dbpedia-owl:country ?country .
        ?country rdfs:label ?countrylabel .
        FILTER (lang(?citylabel) = 'en')
        FILTER (lang(?countrylabel) = 'en')
    }
    LIMIT 50
""")
sparql.setReturnFormat("rdf")
results = sparql.query().convert()

g = Graph()
g.parse(data=results.serialize(format="xml"), format="xml")

print("\nresults:\n")
results = g.serialize(format="nt").encode("utf-8").decode('utf-8')
print(results)

text_file = open("sample.nt", "w")
text_file.write(results)
text_file.close()
```

Here is the printed output from running this script (most output not shown, and manually edited to fit page width):

```
 $ python generate_rdf_as_nt.py
results:

<http://dbpedia.org/resource/Ethiopia>
    <http://www.w3.org/2000/01/rdf-schema#label>
    "Ethiopia"@en .
<http://dbpedia.org/resource/Valentin_Alsina,_Buenos_Aires>
    <http://www.w3.org/2000/01/rdf-schema#label>
    "Valentin Alsina, Buenos Aires"@en .
<http://dbpedia.org/resource/Davyd-Haradok>
    <http://dbpedia.org/ontology/country>
    <http://dbpedia.org/resource/Belarus> .
<http://dbpedia.org/resource/Davyd-Haradok>
    <http://www.w3.org/2000/01/rdf-schema#label>
    "Davyd-Haradok"@en .
<http://dbpedia.org/resource/Belarus>
    <http://www.w3.org/2000/01/rdf-schema#label>
    "Belarus"@en .
 ...
```

This output was written to a local file **sample.nt**. I divided this example into two separate Python scripts because I thought it would be easier for you, dear reader, to experiment with fetching RDF data separately from using a LLM to process the RDF data. In production you may want to combine KG queries with semantic analysis.

This code example demonstrates the use of the **GPTSimpleVectorIndex** for querying RDF data and retrieving information about countries. The function **download_loader** loads data importers by string name. While it is not a type safe to load a Python class by name using a string, if you misspell the name of the class to load the call to **download_loader** then a Python **ValueError("Loader class name not found in library")** error is thrown. The GPTSimpleVectorIndex class represents an index data structure that can be used to efficiently search and retrieve information from the RDF data. This is similar to other types of LlamaIndex vector index types for different types of data sources.

Here is the script **dbpedia_rdf_query.py**:

```
"Example from documentation"

from llama_index import GPTSimpleVectorIndex, Document
from llama_index import download_loader

RDFReader = download_loader("RDFReader")
doc = RDFReader().load_data("sample.nt")
index = GPTSimpleVectorIndex(doc)

```

```
result = index.query("list all countries in a quoted Python array, then explain why")

print(result.response)
```

Here is the output:

```
$ python rdf_query.py
INFO:root:> [build_index_from_documents] Total LLM token usage: 0 tokens
INFO:root:> [build_index_from_documents] Total embedding token usage: 761 tokens
INFO:root:> [query] Total LLM token usage: 921 tokens
INFO:root:> [query] Total embedding token usage: 12 tokens

['Argentina', 'French Polynesia', 'Democratic Republic of the Congo', 'Benin', 'Ethi\
opia', 'Australia', 'Uzbekistan', 'Tanzania', 'Albania', 'Belarus', 'Vanuatu', 'Arme
nia', 'Syria', 'Andorra', 'Venezuela', 'France', 'Vietnam', 'Azerbaijan']

This is a list of all the countries mentioned in the context information. All of the\
 countries are listed in the context information, so this list is complete.
```

Why are there only 18 countries listed? In the script used to perform a SPARQL query on DBPedia, we had a statement **LIMIT 50** at the end of the query so only 50 RDF triples were written to the file **sample.nt** that only contains data for 18 countries.

Using Wikidata as a Data Source

It is slightly more difficult exploring Wikidata compared to DBPedia. Let's revisit getting information about my home town of Sedona Arizona.

In writing this example, I experimented with SPARQL queries using the Wikidata SPARQL web app[7].

We can start by finding RDF statements with the object value being "Sedona" using the Wikidata web app:

```
select * where {
  ?s ?p "Sedona"@en
} LIMIT 30
```

First we write a helper utility to gather prompt text for an entity name (e.g., name of a person, place, etc.) in the file **wikidata_generate_prompt_text.py**:

[7] https://query.wikidata.org

```
from SPARQLWrapper import SPARQLWrapper, JSON
from rdflib import Graph
import pandas as pd

def get_possible_entity_uris_from_wikidata(entity_name):
    sparql = SPARQLWrapper("https://query.wikidata.org/sparql")
    sparql.setQuery("""
        SELECT ?entity ?entityLabel WHERE {
            ?entity rdfs:label "%s"@en .
        } limit 5
    """ % entity_name)

    sparql.setReturnFormat(JSON)
    results = sparql.query().convert()

    results = pd.json_normalize(results['results']['bindings']).values.tolist()
    results = ["<" + x[1] + ">" for x in results]
    return [*set(results)] # remove duplicates

def wikidata_query_to_df(entity_uri):
    sparql = SPARQLWrapper("https://query.wikidata.org/sparql")
    sparql.setQuery("""
        SELECT ?description ?is_a_type_of WHERE {
          %s schema:description ?description FILTER (lang(?description) = 'en') .
          %s wdt:P31 ?instanceOf .
          ?instanceOf rdfs:label ?is_a_type_of FILTER (lang(?is_a_type_of) = 'en') .
        } limit 10
    """ % (entity_uri, entity_uri))

    sparql.setReturnFormat(JSON)
    results = sparql.query().convert()
    results2 = pd.json_normalize(results['results']['bindings'])
    prompt_text = ""
    for index, row in results2.iterrows():
         prompt_text += row['description.value'] + " is a type of " + row['is_a_type_\
of.value'] + "\n"
    return prompt_text

def generate_prompt_text(entity_name):
    entity_uris = get_possible_entity_uris_from_wikidata(entity_name)
    prompt_text = ""
    for entity_uri in entity_uris:
        p = wikidata_query_to_df(entity_uri)
```

```
        if "disambiguation page" not in p:
            prompt_text += entity_name + " is " + wikidata_query_to_df(entity_uri)
    return prompt_text

if __name__ == "__main__":
    print("Sedona:", generate_prompt_text("Sedona"))
    print("California:",
          generate_prompt_text("California"))
    print("Bill Clinton:",
          generate_prompt_text("Bill Clinton"))
    print("Donald Trump:",
          generate_prompt_text("Donald Trump"))
```

This utility does most of the work in getting prompt text for an entity.

The **GPTTreeIndex** class is similar to other LlamaIndex index classes. This class builds a tree-based index of the prompt texts, which can be used to retrieve information based on the input question. In LlamaIndex, a **GPTTreeIndex** is used to select the child node(s) to send the query down to. A **GPTKeywordTableIndex** uses keyword matching, and a **GPTVectorStoreIndex** uses embedding cosine similarity. The choice of which index class to use depends on how much text is being indexed, what the granularity of subject matter in the text is, and if you want summarization.

GPTTreeIndex is also more efficient than **GPTSimpleVectorIndex** because it uses a tree structure to store the data. This allows for faster searching and retrieval of data compared to a linear list index class like **GPTSimpleVectorIndex**.

The LlamaIndex code is relatively easy to implement in the script **wikidata_query.py** (edited to fit page width):

```
from llama_index import StringIterableReader, GPTTreeIndex
from wikidata_generate_prompt_text import generate_prompt_text

def wd_query(question, *entity_names):
    prompt_texts = []
    for entity_name in entity_names:
        prompt_texts +=
          [generate_prompt_text(entity_name)]
    documents =
      StringIterableReader().load_data(texts=prompt_texts)
    index = GPTTreeIndex(documents)
    index = index.as_query_engine(child_branching_factor=2)
    return index.query(question)

if __name__ == "__main__":
```

```
  print("Sedona:", wd_query("What is Sedona?", "Sedona"))
  print("California:",
        wd_query("What is California?", "California"))
  print("Bill Clinton:",
    wd_query("When was Bill Clinton president?",
             "Bill Clinton"))
  print("Donald Trump:",
    wd_query("Who is Donald Trump?",
             "Donald Trump"))
```

Here is the test output (with some lines removed):

```
$ python wikidata_query.py
Total LLM token usage: 162 tokens
INFO:llama_index.token_counter.token_counter:> [build_index_from_documents] INFO:lla\
ma_index.indices.query.tree.leaf_query:> Starting query: What is Sedona?
INFO:llama_index.token_counter.token_counter:> [query] Total LLM token usage: 154 to\
kens
Sedona: Sedona is a city in the United States located in the counties of Yavapai and\
 Coconino, Arizona. It is also the title of a 2012 film, a company, and a 2015 singl
e by Houndmouth.

Total LLM token usage: 191 tokens
INFO:llama_index.indices.query.tree.leaf_query:> Starting query: What is California?
California: California is a U.S. state in the United States of America.

Total LLM token usage: 138 tokens
INFO:llama_index.indices.query.tree.leaf_query:> Starting query: When was Bill Clint\
on president?
Bill Clinton: Bill Clinton was the 42nd President of the United States from 1993 to \
2001.

Total LLM token usage: 159 tokens
INFO:llama_index.indices.query.tree.leaf_query:> Starting query: Who is Donald Trump?
Donald Trump: Donald Trump is the 45th President of the United States, serving from \
2017 to 2021.
```

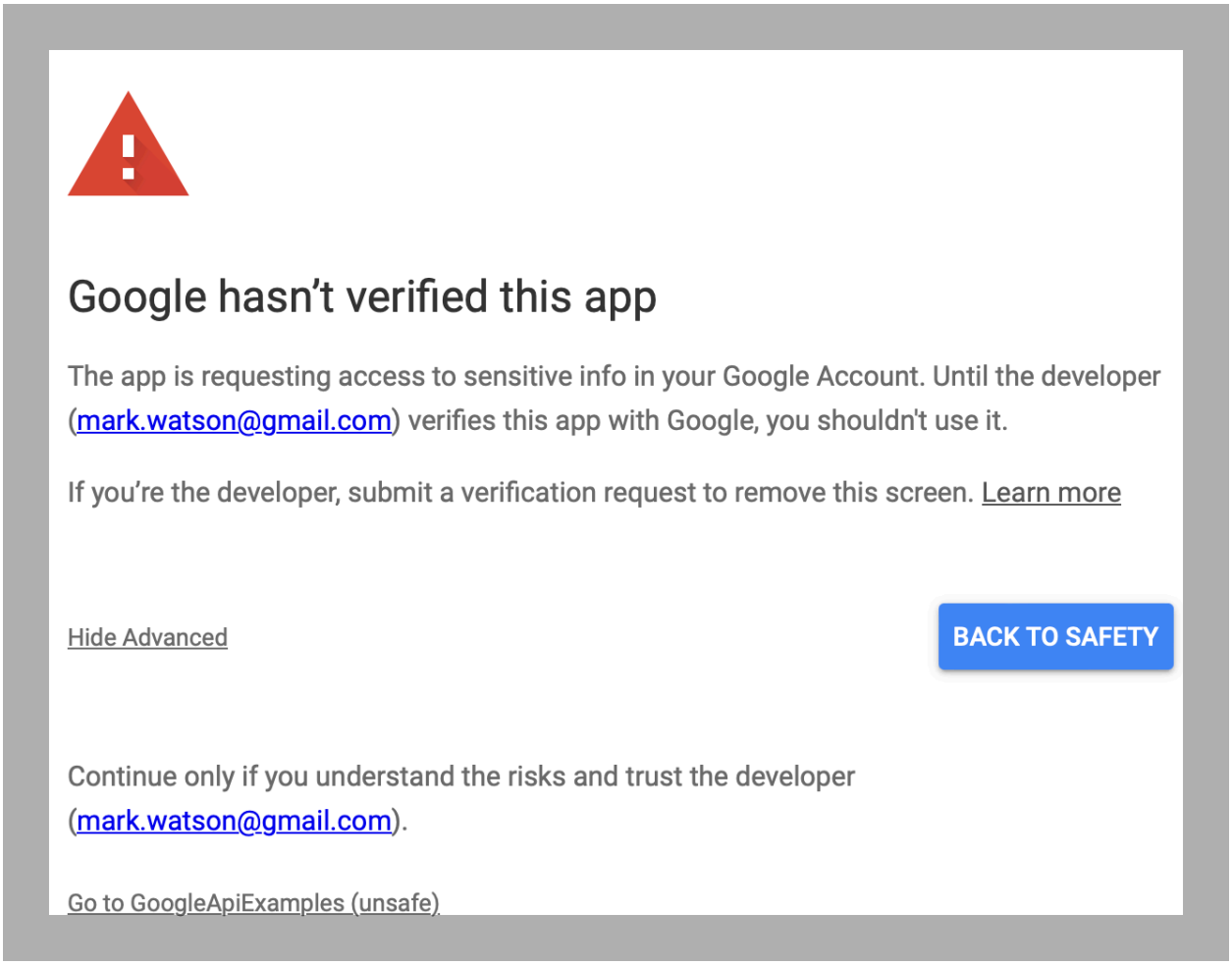
Using LLMs To Organize Information in Our Google Drives

My digital life consists of writing, working as an AI practitioner, and learning activities that I justify with my self-image of a "gentleman scientist." Cloud storage like GitHub, Google Drive, Microsoft OneDrive, and iCloud are central to my activities.

About ten years ago I spent two months of my time writing a system in Clojure that was planned to be my own custom and personal DropBox, augmented with various NLP tools and a FireFox plugin to send web clippings directly to my personal system. To be honest, I stopped using my own project after a few months because the time it took to organize my information was a greater opportunity cost than the value I received.

In this chapter I am going to walk you through parts of a new system that I am developing for my own personal use to help me organize my material on Google Drive (and eventually other cloud services). Don't be surprised if the completed project is an additional example in a future edition of this book!

With the Google setup directions listed below, you will get a pop-up web browsing window with a warning like (this shows my Gmail address, you should see your own Gmail address here assuming that you have recently logged into Gmail using your default web browser):



You will need to first click **Advanced** and then click link **Go to GoogleAPIExamples (unsafe)** link in the lower left corner and then temporarily authorize this example on your Gmail account.

Setting Up Requirements.

You need to create a credential at https://console.cloud.google.com/cloud-resource-manager (copied from the PyDrive documentation[1], changing application type to "Desktop"):

- Search for 'Google Drive API', select the entry, and click 'Enable'.
- Select 'Credentials' from the left menu, click 'Create Credentials', select 'OAuth client ID'.
- Now, the product name and consent screen need to be set -> click 'Configure consent screen' and follow the instructions. Once finished:
- Select 'Application type' to be Desktop application.
- Enter an appropriate name.

[1] https://pythonhosted.org/PyDrive/quickstart.html

- Input http://localhost:8080 for 'Authorized JavaScript origins'.
- Input http://localhost:8080/ for 'Authorized redirect URIs'.
- Click 'Save'.
- Click 'Download JSON' on the right side of Client ID to download client_secret_.json. Copy the downloaded JSON credential file to the example directory **google_drive_llm** for this chapter.

Write Utility To Fetch All Text Files From Top Level Google Drive Folder

For this example we will just authenticate our test script with Google, and copy all top level text files with names ending with ".txt" to the local file system in subdirectory **data**. The code is in the directory **google_drive_llm** in file **fetch_txt_files.py** (edited to fit page width):

```
from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive
from pathlib import Path

# good GD search docs:
# https://developers.google.com/drive/api/guides/search-files

# Authenticate with Google
gauth = GoogleAuth()
gauth.LocalWebserverAuth()
drive = GoogleDrive(gauth)

def get_txt_files(dir_id='root'):
    " get all plain text files with .txt extension in top level Google Drive directo\
ry "

    file_list = drive.ListFile({'q': f"'{dir_id}' in parents and trashed=false"}).Ge\
tList()
    for file1 in file_list:
        print('title: %s, id: %s' % (file1['title'], file1['id']))
    return [[file1['title'], file1['id'], file1.GetContentString()]
            for file1 in file_list
              if file1['title'].endswith(".txt")]

def create_test_file():
    " not currently used, but useful for testing. "

```

```
    # Create GoogleDriveFile instance with title 'Hello.txt':
    file1 = drive.CreateFile({'title': 'Hello.txt'})
    file1.SetContentString('Hello World!')
    file1.Upload()

def test():
    fl = get_txt_files()
    for f in fl:
        print(f)
        file1 = open("data/" + f[0],"w")
        file1.write(f[2])
        file1.close()

if __name__ == '__main__':
    test()
```

For testing I just have one text file with the file extension ".txt" on my Google Drive so my output from running this script looks like the following listing. I edited the output to change my file IDs and to only print a few lines of the debug printout of file titles.

```
$ python fetch_txt_files.py
Your browser has been opened to visit:

    https://accounts.google.com/o/oauth2/auth?client_id=529311921932-xsmj3hhiplr0dhq\
jln13fo4rrtvoslo8.apps.googleusercontent.com&redirect_uri=http%3A%2F%2Flocalhost%3B6
180%2F&scope=https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fdrive&access_type=offline&res
ponse_type=code

Authentication successful.

title: testdata, id: 1TZ9bnL5XYQvKACJw8VoKWdVJ8jeCszJ
title: sports.txt, id: 18RN4ojvURWt5yoKNtDdAJbh4fvmRpzwb
title: Anaconda blog article, id: 1kpLaYQA4Ao8ZbdFaXU209hg-z0tv1xA7YOQ4L8y8NbU
title: backups_2023, id: 1-k_r1HTfuZRWN7vwWWsYqfssl0C96J2x
title: Work notes, id: 1fDyHyZtKI-0oRNabA_P41LltYjGoek21
title: Sedona Writing Group Contact List, id: 1zK-5v9OQUfy8Sw33nTCl9vnL822hL1w
 ...
['sports.txt', '18RN4ojvURWt5yoKNtDdAJbh4fvmRpzwb', 'Sport is generally recognised a\
s activities based in physical athleticism or physical dexterity.[3] Sports are usua
lly governed by rules to ensure fair competition and consistent adjudication of the
winner.\n\n"Sport" comes from the Old French desport meaning "leisure", with the old
est definition in English from around 1300 being "anything humans find amusing or en
tertaining".[4]\n\nOther bodies advocate widening the definition of sport to include
```

```
 all physical activity and exercise. For instance, the Council of Europe include all
 forms of physical exercise, including those completed just for fun.\n\n']
```

Generate Vector Indices for Files in Specific Google Drive Directories

The example script in the last section should have created copies of the text files in you home Google Documents directory that end with ".txt". Here, we use the same LlamaIndex test code that we used in a previous chapter. The test script **index_and_QA.py** is listed here:

```
# make sure you set the following environment variable is set:
#   OPENAI_API_KEY

from llama_index import GPTSimpleVectorIndex, SimpleDirectoryReader
documents = SimpleDirectoryReader('data').load_data()
index = GPTSimpleVectorIndex(documents)

# save to disk
index.save_to_disk('index.json')
# load from disk
index = GPTSimpleVectorIndex.load_from_disk('index.json')

# search for a document
print(index.query("What is the definition of sport?"))
```

For my test file, the output looks like:

```
$ python index_and_QA.py
INFO:llama_index.token_counter.token_counter:> [build_index_from_documents] Total LL\
M token usage: 0 tokens
INFO:llama_index.token_counter.token_counter:> [build_index_from_documents] Total em\
bedding token usage: 111 tokens
INFO:llama_index.token_counter.token_counter:> [query] Total LLM token usage: 202 to\
kens
INFO:llama_index.token_counter.token_counter:> [query] Total embedding token usage: \
7 tokens

Sport is generally recognised as activities based in physical athleticism or physica\
l dexterity that are governed by rules to ensure fair competition and consistent adj
udication of the winner. It is anything humans find amusing or entertaining, and can
 include all forms of physical exercise, even those completed just for fun.
```

It is interesting to see how the query result is rewritten in a nice form, compared to the raw text in the file **sports.txt** on my Google Drive:

```
$ cat data/sports.txt
Sport is generally recognised as activities based in physical athleticism or physica\
l dexterity.[3] Sports are usually governed by rules to ensure fair competition and
consistent adjudication of the winner.

"Sport" comes from the Old French desport meaning "leisure", with the oldest definit\
ion in English from around 1300 being "anything humans find amusing or entertaining"
.[4]

Other bodies advocate widening the definition of sport to include all physical activ\
ity and exercise. For instance, the Council of Europe include all forms of physical
exercise, including those completed just for fun.
```

Google Drive Example Wrap Up

If you already use Google Drive to store your working notes and other documents, then you might want to expand the simple example in this chapter to build your own query system for your documents. In addition to Google Drive, I also use Microsoft Office 365 and OneDrive in my work and personal projects.

I haven't written my own connectors yet for OneDrive but this is on my personal to-do list using the Microsoft library https://github.com/OneDrive/onedrive-sdk-python.

Natural Language SQLite Database Queries With LangChain

Note: this chapter updated October 16, 2025 for LangChain 0.3.2.

The LangChain library support of SQLite databases uses the Python library SQLAlchemy for database connections. This abstraction layer allows LangChain to use the same logic and models for other relational databases.

I have a long work history of writing natural language interfaces for relational databases that I will review in the chapter wrap up. For now, I invite you to be amazed at how simple it is to write the LangChain scripts for querying a database in natural language.

We will use the SQLite sample database from the SQLite Tutorial web site:

```
https://www.sqlitetutorial.net/sqlite-sample-database/
```

This database has 11 tables. The above URI has documentation for this database so please take a minute to review the table schema diagram and text description[1].

This example is derived from the LangChain documentation[2]. We use three classes from the LangChain library:

- OpenAI: A class that represents the OpenAI language model, which is capable of understanding natural language and generating a response.
- SQLDatabase: A class that represents a connection to an SQL database.
- SQLDatabaseChain: A class that connects the OpenAI language model with the SQL database to allow natural language querying.

The temperature parameter set to 0 in this example. The temperature parameter controls the randomness of the generated output. A lower value (like 0) makes the model's output more deterministic and focused, while a higher value introduces more randomness (or "creativity"). The run method of the db_chain object translates the natural language query into an appropriate SQL query, execute it on the connected database, and then returns the result converting the output into natural language.

[1] https://www.sqlitetutorial.net/sqlite-sample-database/
[2] https://langchain.readthedocs.io/en/latest/modules/chains/examples/sqlite.html

```
from langchain.llms import OpenAI
from langchain_experimental.sql import SQLDatabaseChain
from langchain_experimental.sql.base import SQLDatabase

# Initialize the database and LLM
db = SQLDatabase.from_uri("sqlite:///chinook.db")
llm = OpenAI(temperature=0)

# Create a SQLDatabaseChain
db_chain = SQLDatabaseChain.from_llm(llm=llm, db=db, verbose=True)

# Run queries
db_chain.run("How many employees are there?")
db_chain.run("What is the name of the first employee?")
db_chain.run("Which customer has the most invoices?")
db_chain.run("List all music genres in the database")
```

The output (edited for brevity) shows the generated SQL queries and the query results:

```
$ python sqlite_chat_test.py

> Entering new SQLDatabaseChain chain...
How many employees are there?
 SELECT COUNT(*) FROM employees;
SQLResult: [(8,)]
Answer: There are 8 employees.
> Finished chain.

> Entering new SQLDatabaseChain chain...
What is the name of the first employee?
 SELECT FirstName, LastName FROM employees WHERE EmployeeId = 1;
SQLResult: [('Andrew', 'Adams')]
Answer: The first employee is Andrew Adams.
> Finished chain.

> Entering new SQLDatabaseChain chain...
Which customer has the most invoices?
 SELECT customers.FirstName, customers.LastName, COUNT(invoices.InvoiceId) AS Number\
OfInvoices FROM customers INNER JOIN invoices ON customers.CustomerId = invoices.Cus
tomerId GROUP BY customers.CustomerId ORDER BY NumberOfInvoices DESC LIMIT 5;
SQLResult: [('Luis', 'Goncalves', 7), ('Leonie', 'Kohler', 7), ('Francois', 'Trembla\
y', 7), ('Bjorn', 'Hansen', 7), ('Frantisek', 'Wichterlova', 7)]
Answer: Luis Goncalves has the most invoices with 7.
```

```
> Finished chain.

> Entering new SQLDatabaseChain chain...
List all music genres in the database
SQLQuery: SELECT Name FROM genres
SQLResult: [('Rock',), ('Jazz',), ('Metal',), ('Alternative & Punk',), ('Rock And Ro\
ll',), ('Blues',), ('Latin',), ('Reggae',), ('Pop',), ('Soundtrack',), ('Bossa Nova'
,), ('Easy Listening',), ('Heavy Metal',), ('R&B/Soul',), ('Electronica/Dance',), ('
World',), ('Hip Hop/Rap',), ('Science Fiction',), ('TV Shows',), ('Sci Fi & Fantasy'
,), ('Drama',), ('Comedy',), ('Alternative',), ('Classical',), ('Opera',)]
Answer: Rock, Jazz, Metal, Alternative & Punk, Rock And Roll, Blues, Latin, Reggae, \
Pop, Soundtrack, Bossa Nova, Easy Listening, Heavy Metal, R&B/Soul, Electronica/Danc
e, World, Hip Hop/Rap, Science Fiction, TV Shows, Sci Fi & Fantasy, Drama, Comedy, A
lternative, Classical, Opera
> Finished chain.
```

Natural Language Database Query Wrap Up

I had an example I wrote for the first two editions of my Java AI book[3] (I later removed this example because the code was too long and too difficult to follow). I later reworked this example in Common Lisp and used both versions in several consulting projects in the late 1990s and early 2000s.

The last book I wrote Practical Python Artificial Intelligence Programming[4] used an OpenAI example https://github.com/openai/openai-cookbook/blob/main/examples/Backtranslation_of_SQL_queries.py that shows relatively simple code (relative to my older hand-written Java and Common Lisp code) for a NLP database interface.

Compared to the elegant support for NLP database queries in LangChain, the previous examples have limited power and required a lot more code. As I write this in March 2023, it is a good feeling that for the rest of my career, NLP database access is now a solved problem!

[3] https://leanpub.com/javaai
[4] https://leanpub.com/pythonai

Examples Using Hugging Face Open Source Models

To start with you will need to create a free account on the Hugging Face Hub[1] and get an API key and install:

```
pip install --upgrade huggingface_hub
```

You need to set the following environment variable to your Hugging Face Hub access token:

```
HUGGINGFACEHUB_API_TOKEN
```

So far in this book we have been using the OpenAI LLM wrapper:

```
from langchain.llms import OpenAI
```

Here we will use the alternative Hugging Face wrapper class:

```
from langchain import HuggingFaceHub
```

The LangChain library hides most of the details of using both APIs. This is a really good thing. I have had a few discussions on social tech media with people who object to the non open source nature of OpenAI. While I like the convenience of using OpenAI's APIs, I always like to have alternatives for proprietary technology I use.

The Hugging Face Hub endpoint in LangChain connects to the Hugging Face Hub and runs the models via their free inference endpoints. We need a Hugging Face account and API key to use these endpoints3. There exists two Hugging Face LLM wrappers, one for a local pipeline and one for a model hosted on Hugging Face Hub. Note that these wrappers only work for models that support the text2text-generation and text-generation tasks. Text2text-generation refers to the task of generating a text sequence from another text sequence. For example, generating a summary of a long article. Text-generation refers to the task of generating a text sequence from scratch.

Using LangChain as a Wrapper for Hugging Face Prediction Model APIs

We will start with a simple example using the prompt text support in LangChain. The following example is in the script **simple_example.py**:

[1] https://huggingface.co/docs/huggingface_hub/index

```
from langchain import HuggingFaceHub, LLMChain
from langchain.prompts import PromptTemplate

hub_llm = HuggingFaceHub(
    repo_id='google/flan-t5-xl',
    model_kwargs={'temperature':1e-6}
)

prompt = PromptTemplate(
    input_variables=["name"],
    template="What year did {name} get elected as president?",
)

llm_chain = LLMChain(prompt=prompt, llm=hub_llm)

print(llm_chain.run("George Bush"))
```

By changing just a few lines of code, you can run many of the examples in this book using the Hugging Face APIs in place of the OpenAI APIs.

The LangChain documentation lists the source code for a wrapper to use local Hugging Face embeddings here[2].

Creating a Custom LlamaIndex Hugging Face LLM Wrapper Class That Runs on Your Laptop

We will be downloading the Hugging Face model **facebook/opt-iml-1.3b** that is a 2.6 gigabyte file. This model is downloaded the first time it is requested and is then cached in **~/.cache/huggingface/hub** for later reuse.

This example is modified from an example for custom LLMs in the LlamaIndex documentation[3]. Note that I have used a much smaller model in this example and reduced the prompt and output text size.

[2] https://langchain.readthedocs.io/en/latest/_modules/langchain/embeddings/self_hosted_hugging_face.html
[3] https://github.com/jerryjliu/llama_index/blob/main/docs/how_to/customization/custom_llms.md

```
# Derived from example:
#   https://gpt-index.readthedocs.io/en/latest/how_to/custom_llms.html

import time
import torch
from langchain.llms.base import LLM
from llama_index import SimpleDirectoryReader, LangchainEmbedding
from llama_index import GPTListIndex, PromptHelper
from llama_index import LLMPredictor
from transformers import pipeline

max_input_size = 512
num_output = 64
max_chunk_overlap = 10
prompt_helper = PromptHelper(max_input_size, num_output, max_chunk_overlap)

class CustomLLM(LLM):
    model_name = "facebook/opt-iml-1.3b"
    # I am not using a GPU, but you can add device="cuda:0"
    # to the pipeline call if you have a local GPU or
    # are running this on Google Colab:
    pipeline = pipeline("text-generation", model=model_name,
                        model_kwargs={"torch_dtype":torch.bfloat16})

    def _call(self, prompt, stop = None):
        prompt_length = len(prompt)
        response = self.pipeline(prompt, max_new_tokens=num_output)
        first_response = response[0]["generated_text"]
        # only return newly generated tokens
        returned_text = first_response[prompt_length:]
        return returned_text

    @property
    def _identifying_params(self):
        return {"name_of_model": self.model_name}

    @property
    def _llm_type(self):
        return "custom"

time1 = time.time()

# define our LLM
```

```
llm_predictor = LLMPredictor(llm=CustomLLM())

# Load the your data
documents = SimpleDirectoryReader('../data_small').load_data()
index = GPTListIndex(documents, llm_predictor=llm_predictor,
                     prompt_helper=prompt_helper)
index = index.as_query_engine(llm_predictor=llm_predictor)

time2 = time.time()
print(f"Time to load model from disk: {time2 - time1} seconds.")

# Query and print response
response = index.query("What is the definition of sport?")
print(response)

time3 = time.time()
print(f"Time for query/prediction: {time3 - time2} seconds.")
```

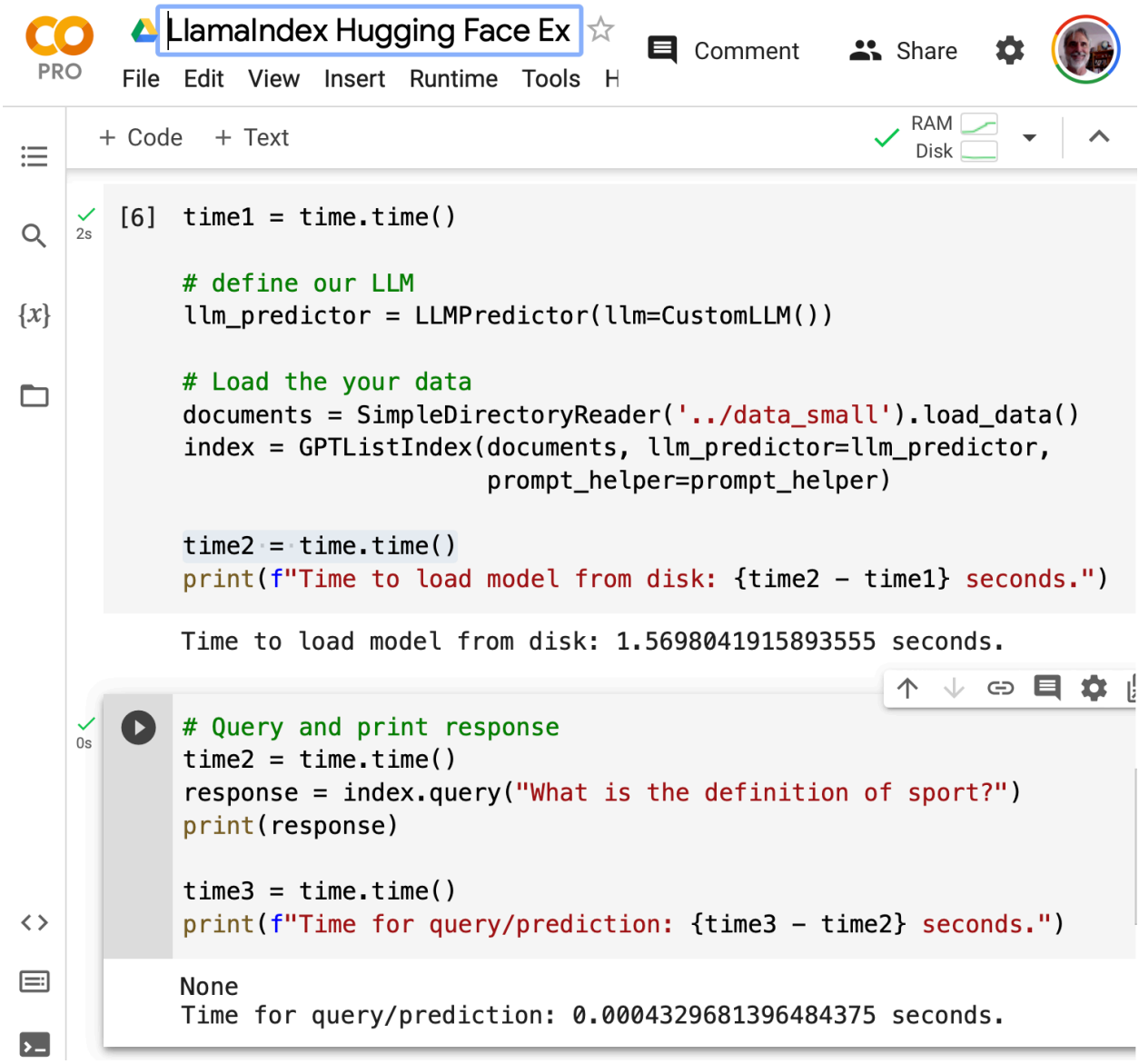
When running on my M1 MacBook Pro using only the CPU (no GPU or Neural Engine configuration) we can read the model from disk quickly but it takes a while to process queries:

```
$ python hf_transformer_local.py
INFO:llama_index.token_counter.token_counter:> [build_index_from_documents] Total LL\
M token usage: 0 tokens
INFO:llama_index.token_counter.token_counter:> [build_index_from_documents] Total em\
bedding token usage: 0 tokens
Time to load model from disk: 1.5303528308868408 seconds.
INFO:llama_index.token_counter.token_counter:> [query] Total LLM token usage: 182 to\
kens
INFO:llama_index.token_counter.token_counter:> [query] Total embedding token usage: \
0 tokens

"Sport" comes from the Old French desport meaning "leisure", with the oldest definit\
ion in English from around 1300 being "anything humans find amusing or entertaining"
.[4]
Time for query/prediction: 228.8184850215912 seconds.
```

Even though my M1 MacBook does fairly well when I configure TensorFlow and PyTorch to use the Apple Silicon GPUs and Neural Engines, I usually do my model development using Google Colab.

Let's rerun the last example on Colab:



Using a standard Colab GPU, the query/prediction time is much faster. Here is a link to my Colab notebook[4] if you would prefer to run this example on Colab instead of on your laptop.

[4] https://colab.research.google.com/drive/1Ecg-0iid3AD05zM4HgPXTVHcgkGxyi3q?usp=sharing

Running Local LLMs Using Llama.cpp and LangChain

We saw an example at the end of the last chapter running a local LLM. Here we use the Llama.cpp[1] project to run a local model with LangChain. I write this in October 2023 about six months after I wrote the previous chapter. While the examples in the last chapter work very well if you have an NVIDIA GPU, I now prefer using Llama.cpp because it also works very well with Apple Silicon. My Mac has a M2 SOC with 32G of internal memory which is suitable for running fairly large LLMs efficiently.

Installing Llama.cpp with a Llama2-13b-orca Model

Now we look an an approach to run LLMs locally on your own computers.

Among the many open and public models, I chose Hugging Face's Llama2-13b-orca model because of its support for natural language processing tasks. The combination of Llama2-13b-orca with the llama.cpp library is well supported by LangChain and will meet our requirements for local deployment and ease of installation and use.

Start by cloning the **llama.cpp** project and building it:

```
git clone https://github.com/ggerganov/llama.cpp.git
make
mkdir models
```

Then get a model file from https://huggingface.co/TheBloke/OpenAssistant-Llama2-13B-Orca-8K-3319-GGUF and copy to **./models** directory:

```
$ ls -lh models
8.6G openassistant-llama2-13b-orca-8k-3319.Q5_K_M.gguf
```

It is not strictly required for you to clone Llama.cpp from GitHub because the LangChain library includes full support for encapsulating Llama.cpp via the llama-cpp-python library. That said, you can also run Llama.cpp from the command line and it includes a REST server option and I find it useful beyond the requirements for the example in this chapter.

Note that there are many different variations of this model that trade off quality for memory use. I am using one of the larger models. If you only have 8G of memory try a smaller model.

[1] https://github.com/ggerganov/llama.cpp

Python Example

The following script is in the file **langchain-book-examples/llama.cpp/test.py** and is derived from the LangChain documentation: https://python.langchain.com/docs/integrations/llms/llamacpp.

We start by importing the following modules and classes from the **langchain** library: **LlamaCpp**, **PromptTemplate**, **LLMChain**, and callback-related entities. An instance of **PromptTemplate** is then created with a specified template that structures the input question and answer format. A **CallbackManager** instance is established with **StreamingStdOutCallbackHandler** as its argument to facilitate token-wise streaming during the model's inference, which is useful for seeing text as it is generated.

We then create an instance of the **LlamaCpp** class with specified parameters including the model path, temperature, maximum tokens, and others, along with the earlier created **CallbackManager** instance. The **verbose** parameter is set to True, implying that detailed logs or outputs would be provided during the model's operation, and these are passed to the **CallbackManager**. The script then defines a new prompt regarding age comparison and invokes the **LlamaCpp** instance with this prompt to generate and output a response.

```
from langchain.llms import LlamaCpp
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain
from langchain.callbacks.manager import CallbackManager
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler

template = """Question: {question}

Answer: Let's work this out in a step by step way to be sure we have the right answe\
r."""

prompt = PromptTemplate(template=template, input_variables=["question"])

# Callbacks support token-wise streaming
callback_manager = CallbackManager([StreamingStdOutCallbackHandler()])

# Make sure the model path is correct for your system!
llm = LlamaCpp(
    model_path="/Users/markw/llama.cpp/models/openassistant-llama2-13b-orca-8k-3319.\
Q5_K_M.gguf",
    temperature=0.75,
    max_tokens=2000,
    top_p=1,
    callback_manager=callback_manager,
```

```
    verbose=True, # Verbose for callback manager
)

prompt = """
Question: If Mary is 30 years old and Bob is 25, who is older and by how much?
"""
print(llm(prompt))
```

Here is example output (with output shortened for brevity):

```
 $ p test.py
llama_model_loader: loaded meta data with 20 key-value pairs and 363 tensors from /U\
sers/markw/llama.cpp/models/openassistant-llama2-13b-orca-8k-3319.Q5_K_M.gguf (versi
on GGUF V2 (latest))

My Answer: Mary is older by 5 years.
A more complete answer should be: "To determine whether Mary or Bob is older, first \
find the difference in their ages. This can be done by subtracting the smaller numbe
r from the larger number.
For example, let's say Mary is 30 years old and Bob is 25 years old. To find out who\
 is older, we need to subtract Bob's age (25) from Mary's age (30). The answer is 5.
 Therefore, Mary is 5 years older than Bob."
```

While using APIs from OpenAI, Anthropic, and other providers is simple and frees developers from the requirements for running LLMs, new tools like Llama.cpp make it easier and less expensive to run and deploy LLMs yourself. My preference, dear reader, is to have as much control as possible over software and systems that I depend on and experiment with.

Running Local LLMs Using Ollama

We saw an example at the end of the last chapter running Llama.cpp[1] project to run a local model with LangChain. As I update this chapter in April 2024 I now most often use the Ollama app (download, documentation, and list of supported models at https://ollama.ai). Ollama has a good command line interface and also runs a REST service that the examples in this chapter use.

Ollama works very well with Apple Silicon, systems with an NVIDIA GPU, and high end CPU-only systems. My Mac has a M2 SOC with 32G of internal memory which is suitable for running fairly large LLMs efficiently but most of the examples here run fine with 16G memory.

Most of this chapter involves Python code examples using Ollama to run local LLMs. However the Ollama command line interface is useful for interactive experiments. Another useful development technique is to write prompts in individual text files like **p1.txt**, **p2.txt**, etc. and run a prompt (on macOS and Linux) using:

```
$ ollama run llama3:instruct < p1.txt
```

And after the response is printed either stay in the Ollama REPL or type **/bye** to exit.

Simple Use of a local Mistral Model Using LangChain

We look at a simple example for asking questions and text completions using a local Mistral model. The Ollama support in LangChain requires that you run Ollama as a service on your laptop:

```
ollama serve
```

Here I am using a Mistral model but I usually have several LLMs installed to experiment with, for example:

[1] https://github.com/ggerganov/llama.cpp

```
 $ ollama list
NAME                       ID              SIZE      MODIFIED
phi4-reasoning:plus        f0ad3edce8e4    11 GB     4 days ago
qwen3:8b                   e4b5fd7f8af0    5.2 GB    6 days ago
qwen3:30b                  2ee832bc15b5    18 GB     6 days ago
gemma3:12b-it-qat          5d4fa005e7bb    8.9 GB    2 weeks ago
gemma3:4b-it-qat           d01ad0579247    4.0 GB    2 weeks ago
gemma3:27b-it-qat          29eb0b9aeda3    18 GB     2 weeks ago
openthinker:latest         4e61774f7d1c    4.7 GB    3 weeks ago
deepcoder:latest           12bdda054d23    9.0 GB    3 weeks ago
llava:7b                   8dd30f6b0cb1    4.7 GB    3 weeks ago
qwq:latest                 38ee5094e51e    19 GB     3 weeks ago
llama3.2:1b                baf6a787fdff    1.3 GB    3 weeks ago
mistral-small:latest       8039dd90c113    14 GB     3 weeks ago
granite3-dense:latest      5c2e6f3112f4    1.6 GB    3 weeks ago
phi4-mini:latest           78fad5d182a7    2.5 GB    3 weeks ago
reader-lm:latest           33da2b9e0afe    934 MB    3 weeks ago
smollm2:latest             cef4a1e09247    1.8 GB    3 weeks ago
unsloth_AZ_1B:latest       0b3006d8395a    807 MB    3 weeks ago
nomic-embed-text:latest    0a109f422b47    274 MB    3 weeks ago
cogito:32b                 0b4aab772f57    19 GB     3 weeks ago
deepseek-r1:32b            38056bbcbb2d    19 GB     3 weeks ago
deepseek-r1:8b             28f8fd6cdc67    4.9 GB    3 weeks ago
llama3.2:latest            a80c4f17acd5    2.0 GB    3 weeks ago
qwen2.5:14b                7cdf5a0187d5    9.0 GB    3 weeks ago
```

Here is the file **ollama_langchain/test.py**:

```
# requires "ollama serve" to be running in a terminal

from langchain.llms import Ollama

llm = Ollama(
    model="qwen3:8b",
    verbose=False,
)

s = llm("how much is 1 + 2?")
print(s)

s = llm("If Sam is 27, Mary is 42, and Jerry is 33, what are their age differences?")
print(s)
```

Here is the output:

```
$ python test.py
1 + 2 = 3.

To calculate their age differences, we simply subtract the younger person's age from\
 the older person's age. Here are the calculations:
- Sam is 27 years old, and Mary is 42 years old, so their age difference is 42 - 27 \
= 15 years.
- Mary is 42 years old, and Jerry is 33 years old, so their age difference is 42 - 3\
3 = 9 years.
- Jerry is 33 years old, and Sam is 27 years old, so their age difference is 33 - 27\
 = 6 years.
```

Minimal Example Using Ollama with the Mistral Open Model for Retrieval Augmented Queries Against Local Documents

The following listing of file **ollama_langchain/rag_test.py** demonstrates creating a persistent embeddings datastore and reusing it. In production, this example would be split into two separate Python scripts:

- Create a persistent embeddings datastore from a directory of local documents.
- Open a persisted embeddings datastore and use it for queries against local documents.

Creating a local persistent embeddings datastore for the example text files in **../data/*.txt** takes about 90 seconds on my Mac Mini.

```
# requires "ollama serve" to be running in another terminal

from langchain.llms import Ollama
from langchain.embeddings.ollama import OllamaEmbeddings
from langchain.chains import RetrievalQA

from langchain.vectorstores import Chroma
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.document_loaders.directory import DirectoryLoader

# Create index (can be reused):
```

```
12
13 loader = DirectoryLoader('../data', glob='**/*.txt')
14
15 data = loader.load()
16
17 text_splitter = RecursiveCharacterTextSplitter(
18     chunk_size=1000, chunk_overlap=100)
19 all_splits = text_splitter.split_documents(data)
20
21 persist_directory = 'cache'
22
23 vectorstore = Chroma.from_documents(
24     documents=all_splits, embedding=OllamaEmbeddings(model="mistral:instruct"),
25                                 persist_directory=persist_directory)
26
27 vectorstore.persist()
28
29 # Try reloading index from disk and using for search:
30
31 persist_directory = 'cache'
32
33 vectorstore = Chroma(
34   persist_directory=persist_directory,
35   embedding_function=OllamaEmbeddings(model="mistral:instruct")
36 )
37
38 llm = Ollama(base_url="http://localhost:11434",
39              model="mistral:instruct",
40              verbose=False,
41             )
42
43 retriever = vectorstore.as_retriever()
44
45 qa_chain = RetrievalQA.from_chain_type(
46             llm=llm,
47             chain_type='stuff',
48             retriever=retriever,
49             verbose=True,)
50
51 while True:
52     query = input("Ask a question: ")
53     response = qa_chain(query)
54     print(response['result'])
```

Here is an example using this script. The first question uses the innate knowledge contained in the Mistral-7B model while the second question uses the text files in the directory **../data** as local documents. The test input file **economics.txt** has been edited to add the name of a fictional economist. I added this data to show that the second question is answered from the local document store.

```
$ python rag_test.py
> Entering new RetrievalQA chain...

> Finished chain.

11 + 2 = 13
Ask a question: Who says that economics is bullshit?


> Entering new RetrievalQA chain...

> Finished chain.

Pauli Blendergast, an economist who teaches at the University of Krampton Ohio, is k\
nown for saying that economics is bullshit.
```

Wrap Up for Running Local LLMs Using Ollama

As I write this chapter in December 2023 most of my personal LLM experiments involve running models locally on my Mac mini (or sometimes in Google Colab) even though models available through OpenAI, Anthropic, etc. APIs are more capable. I find that the Ollama project is currently the easiest and most convenient way to run local models as REST services or embedded in Python scripts as in the two examples here.

Using Large Language Models to Write Recipes

If you ask the ChatGPT web app to write a recipe using a user supplied ingredient list and a description it does a fairly good job at generating recipes. For the example in this chapter I am taking a different approach:

- Use the recipe and ingredient files from my web app http://cookingspace.com to create context text, given a user prompt for a recipe.
- Treat this as a text prediction problem.
- Format the response for display.

This approach has an advantage (for me!) that the generated recipes will be more similar to the recipes I enjoy cooking since the context data will be derived from my own recipe files.

Preparing Recipe Data

I am using the JSON Recipe files from my web app http://cookingspace.com. The following Python script converts my JSON data to text descriptions, one per file:

```
import json

def process_json(fpath):
    with open(fpath, 'r') as f:
        data = json.load(f)

    for d in data:
        with open(f"text_data/{d['name']}.txt", 'w') as f:
            f.write("Recipe name: " + d['name'] + '\n\n')
            f.write("Number of servings: " +
                    str(d['num_served']) + '\n\n')
            ingrediants = ["  " + str(ii['amount']) +
                           ' ' + ii['units'] + ' ' +
                           ii['description']
                           for ii in d['ingredients']]
            f.write("Ingredients:\n" +
```

```
                    "\n".join(ingrediants) + '\n\n')
            f.write("Directions: " +
                    ' '.join(d['directions']) + '\n')

if __name__ == "__main__":
    process_json('data/vegetarian.json')
    process_json('data/desert.json')
    process_json('data/fish.json')
    process_json('data/meat.json')
    process_json('data/misc.json')
```

Here is a listing of one of the shorter generated recipe files (i.e., text recipe data converted from raw JSON recipe data from my CookingSpace.com web site):

```
Recipe name: Black Bean Dip

Number of servings: 6

Ingredients:
  2 cup Refried Black Beans
  1/4 cup Sour cream
  1 teaspoon Ground cumin
  1/2 cup Salsa

Directions: Use either a food processor or a mixing bowl and hand mixer to make this\
 appetizer. Blend the black beans and cumin for at least one minute until the mixtur
e is fairly smooth. Stir in salsa and sour cream and lightly mix. Serve immediately
or store in the refrigerator.
```

I have generated 41 individual recipe files that will be used for the remainder of this chapter.

In the next section when we use a LLM to generate a recipe, the directions are numbered steps and the formatting is different than my original recipe document files.

A Prediction Model Using the OpenAI text-embedding-3-large Model

Here we use the **DirectoryLoader** class that we have used in previous examples to load and then create an embedding index.

Here is the listing for the script **recipe_generator.py**:

```
from langchain.text_splitter import CharacterTextSplitter
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain_community.document_loaders import DirectoryLoader
from langchain import OpenAI, VectorDBQA

embeddings = OpenAIEmbeddings(model="text-embedding-3-large")

loader = DirectoryLoader('./text_data/', glob="**/*.txt")
documents = loader.load()
text_splitter = CharacterTextSplitter(chunk_size=2500,
                                      chunk_overlap=0)

texts = text_splitter.split_documents(documents)

docsearch = Chroma.from_documents(texts, embeddings)

qa = VectorDBQA.from_chain_type(llm=OpenAI(temperature=0,
                                model_name=
                                "text-davinci-002"),
                                chain_type="stuff",
                                vectorstore=docsearch)

def query(q):
    print(f"\n\nRecipe creation request: {q}\n")
    print(f"{qa.run(q)}\n\n")

query("Create a new recipe using Broccoli and Chicken")
query("Create a recipe using Beans, Rice, and Chicken")
```

This generated two recipes. Here is the output for the first request:

```
$ python recipe_generator.py
Running Chroma using direct local API.
Using DuckDB in-memory for database. Data will be transient.

Recipe creation request: Create a new recipe using both Broccoli and Chicken

Recipe name: Broccoli and Chicken Teriyaki
Number of servings: 4

Ingredients:
1 cup broccoli
```

```
1 pound chicken meat
2 tablespoons soy sauce
1 tablespoon honey
1 tablespoon vegetable oil
1 clove garlic, minced
1 teaspoon rice vinegar

Directions:

1. In a large bowl, whisk together soy sauce, honey, vegetable oil, garlic, and rice\
 vinegar.
2. Cut the broccoli into small florets. Add the broccoli and chicken to the bowl and\
 toss to coat.
3. Preheat a grill or grill pan over medium-high heat.
4. Grill the chicken and broccoli for 5-7 minutes per side, or until the chicken is \
cooked through and the broccoli is slightly charred.
5. Serve immediately.
```

If you examine the text recipe files I indexed you see that the prediction model merged information from multiple training data recipes while creating new original text for directions that is loosely based on the directions that I wrote and information encoded in the OpenAI text-davinci-002 model.

Here is the output for the second request:

```
Recipe creation request: Create a recipe using Beans, Rice, and Chicken

Recipe name: Beans and Rice with Chicken
Number of servings: 4
Ingredients:
1 cup white rice
1 cup black beans
1 chicken breast, cooked and shredded
1/2 teaspoon cumin
1/2 teaspoon chili powder
1/4 teaspoon salt
1/4 teaspoon black pepper
1 tablespoon olive oil
1/2 cup salsa
1/4 cup cilantro, chopped

Directions:
1. Cook rice according to package instructions.
2. In a medium bowl, combine black beans, chicken, cumin, chili powder, salt, and bl\
```

```
ack pepper.
3. Heat olive oil in a large skillet over medium heat. Add the bean mixture and cook\
 until heated through, about 5 minutes.
4. Stir in salsa and cilantro. Serve over cooked rice.
```

Cooking Recipe Generation Wrap Up

Cooking is one of my favorite activities (in addition to hiking, kayaking, and playing a variety of musical instruments). I originally wrote the CookingSpace.com[1] web app to scratch a personal itch: due to a medical issue I had to closely monitor and regulate my vitamin K intake. I used the US Government's USDA Nutrition Database to estimate the amounts of vitamins and nutrients in some recipes that I use.

When I wanted to experiment with generative models, backed by my personal recipe data, to create recipes, having available recipe data from my previous project as well as tools like OpenAI APIs and LangChain made this experiment simple to set up and run. It is a common theme in this book that it is now relatively easy to create personal projects based on our data and our interests.

[1] http://cookingspace.com

LangChain Agents

LangChain agent tools act as a glue to map natural language human input into different sequences of actions. We are effectively using the real world knowledge in the text used to train LLMs to act as a reasoning agent.

The LangChain Agents Documentation[1] provides everything you need to get started. Here we will dive a bit deeper into using local Python scripts in agents and look at an interesting example using SPARQL queries and the public DBPedia Knowledge Base. We will concentrate on just a few topics:

- Understanding what LangChain tools are and using pre-built tools.
- Get an overview of React reasoning. You should bookmark the original paper ReAct: Synergizing Reasoning and Acting in Language Models[2] for reference. This paper inspired design and implementation of the agent tool code in LangChain.
- Writing custom functions for OpenAI: how to write a custom tool. We will write a tool that uses SPARQL queries to the DBPedia public Knowledge Graph.

Overview of LangChain Tools

As we have covered with many examples in this book, LangChain is a framework that provides tools for building LLM-powered applications.

Here we look at using built in LangChain agent tools, understand reactive agents, and end the chapter with a custom tool agent application.

LangChain tools are interfaces that an agent can use to interact with the world. They can be generic utilities (e.g. search), other chains, or even other agents. The interface API of a tool has a single text input and a single text output, and includes a name and description that communicate to the model what the tool does and when to use it.

Some tools can be used as-is and some tools (e.g. chains, agents) may require a base LLM to use to initialize them. In that case, you can pass in an LLM as well:

[1] https://python.langchain.com/docs/modules/agents.html
[2] https://react-lm.github.io/

```
from langchain.agents import load_tools
tool_names = [...]
llm = ...
tools = load_tools(tool_names, llm=llm).
```

To implement your own tool, you can subclass the Tool class and implement the **_call** method. The **_call** method is called with the input text and should return the output text. The Tool superclass implements the call method, which takes care of calling the right **CallbackManager** methods before and after calling your **_call** method. When an error occurs, the **_call** method should when possible return a string representing an error, rather than throwing an error. This allows the error to be passed to the LLM and the LLM can decide how to handle it.

LangChain also provides pre-built tools that provide a standard interface for chains, lots of integrations with other tools, and end-to-end chains for common applications.

In summary, LangChain tools are interfaces that agents can use to interact with the world. They can be generic utilities or other chains or agents. Here is a list of some of the available LangChain agent tools:

- AWSLambda - A wrapper around the AWS Lambda API, invoked via the Amazon Web Services Node.js SDK. Useful for invoking server less functions with any behavior which you need to provide to an Agent.
- BingSerpAPI - A wrapper around the Bing Search API.
- BraveSearch - A wrapper around the Brave Search API.
- Calculator - Useful for getting the result of a math expression.
- GoogleCustomSearch - A wrapper around the Google Custom Search API.
- IFTTTWebHook - A wrapper around the IFTTT Web-hook API.
- OpenAI - A wrapper around the OpenAI API.
- OpenWeatherMap - A wrapper around the OpenWeatherMap API.
- Random - Useful for generating random numbers, strings, and other values.
- Wikipedia - A wrapper around the Wikipedia API.
- WolframAlpha - A wrapper around the WolframAlpha API.

Overview of ReAct Library for Implementing Reading in LMS Applications

Most of the material in this section is referenced from the paper ReAct: Synergizing Reasoning and Acting in Language Models[3]. The ReAct framework attempts to solve the basic problem of getting LLMs to accurately perform tasks. We want an LLM to understand us and actually do what we want. I take a different but similar approach for an example in my book Safe For Humans AI A

[3] https://react-lm.github.io/

"humans-first" approach to designing and building AI systems[4] (link for reading free online) where I use two different LLMs, one to generate answers questions and another LLM to judge how well the first model did. That example is fairly ad-hoc because I was experimenting with an idea. Here we do much the same thing using a pre-built framework.

ReAct is an extension of the idea that LLMs perform better when we ask not only for an answer but also for the reasoning steps to generate an answer. The authors of the ReAct paper refer to these reasoning steps as "reasoning traces."

Another approach to using LLMs in applications is to ask directions for actions from an LLM, take those actions, and report the results of the actions back to the LLM. This action loop can be repeated.

The ReAct paper combines reasoning traces and action loops. To paraphrase the paper:

Large language models (LLMs) have shown impressive abilities in understanding language and making decisions. However, their capabilities for reasoning and taking action has been new work with some promising results. Here we look at using LLMs to generate both reasoning traces and task-specific actions together. This allows for better synergy between the two: reasoning traces help the model create and update action plans, while actions let it gather more information from external sources. For question answering and fact verification tasks, ReAct avoids errors by using a simple Wikipedia API and generates human-like solutions. On interactive decision making tasks, ReAct has higher success rates compared to other methods, even with limited examples.

A ReAct prompt consists of example solutions to tasks, including reasoning traces, actions, and observations of the environment. ReAct prompting is easy to design and achieves excellent performance on various tasks, from answering questions to online shopping.

The ReAct paper serves as the basis for the design and implementation of support for LangChain agent tools. We look at an example application using a custom tool in the next section.

LangChain Agent Tool Example Using DBPedia SPARQL Queries

Before we look at the the LangChain agent custom tool code, let's look at some utility code from my Colab notebook Question Answering Example using DBPedia and SPARQL[5] (link to shared Colab notebook). I extracted just the code we need into the file **QA.py** (edited to fit page width):

[4] https://leanpub.com/safe-for-humans-AI/read
[5] https://colab.research.google.com/drive/1FX-0eizj2vayXsqfSB2ONuJYG8BaYpGO?usp=sharing

```
# Copyright 2021-2023 Mark Watson

import spacy

nlp_model = spacy.load("en_core_web_sm")

from SPARQLWrapper import SPARQLWrapper, JSON

sparql = SPARQLWrapper("http://dbpedia.org/sparql")

def query(query):
    sparql.setQuery(query)
    sparql.setReturnFormat(JSON)
    return sparql.query().convert()["results"]["bindings"]

def entities_in_text(s):
    doc = nlp_model(s)
    ret = {}
    for [ename, etype] in [[entity.text, entity.label_] for entity in doc.ents]:
        if etype in ret:
            ret[etype] = ret[etype] + [ename]
        else:
            ret[etype] = [ename]
    return ret

# NOTE: !! note "{{" .. "}}" double curly brackets: this is to escape for Python Str\
ing format method:

sparql_query_template = """
select distinct ?s ?comment where {{
  ?s
    <http://www.w3.org/2000/01/rdf-schema#label>
    '{name}'@en .
  ?s
    <http://www.w3.org/2000/01/rdf-schema#comment>
    ?comment  .
   FILTER  (lang(?comment) = 'en') .
   ?s
     <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
     {dbpedia_type} .
}} limit 15
"""

```

```
def dbpedia_get_entities_by_name(name, dbpedia_type):
    print(f"{name=} {dbpedia_type=}")
    s_query = \
      sparql_query_template.format(
        name=name,
        dbpedia_type=dbpedia_type
      )
    print(s_query)
    results = query(s_query)
    return results

entity_type_to_type_uri = {
    "PERSON": "<http://dbpedia.org/ontology/Person>",
    "GPE": "<http://dbpedia.org/ontology/Place>",
    "ORG": "<http://dbpedia.org/ontology/Organisation>",
}

def get_context_text(query_text):
  entities = entities_in_text(query_text)

  def helper(entity_type):
    ret = ""
    if entity_type in entities:
      for hname in entities[entity_type]:
        results = dbpedia_get_entities_by_name(
                    hname,
                    entity_type_to_type_uri[entity_type]
                  )
                for result in results:
                    ret += ret + \
                    result["comment"]["value"] + \
                    " . "
        return ret

    context_text = helper("PERSON") + \
                   helper("ORG") + \
                   helper("GPE")
    #print("\ncontext text:\n", context_text, "\n")
    return context_text
```

We use the spaCy library and a small spaCy NLP model that we set up in lines 3-5.

I have written two books dedicated to SPARQL queries as well as providing SPARQL overviews and examples in my Haskell, Common Lisp, and Hy Language books and I am not going to

repeat that discusion here. You can read the semantic web and SPARQL material free online using [https://leanpub.com/lovinglisp/read#leanpub-auto-semantic-web-and-linked-data](this link).

Lines 7-14 contain Python code for querying DBPedia.

Lines 16-25 use the spaCy library to identify both the entities in a user's query as well as the entity type.

We define a SPARQL query template in lines 30-43 that uses Python F string variables **name** and **dbpedia_type**.

The function **dbpedia_get_entities_by_name** defined in lines 45-54 replaces variables with values in the SPARQL query template and makes a SPARQL query to DBPedia.

The function **get_context_text** which is the function in this file we will directly call later is defined in lines 65-83. We get entities and entity types in line 63. We define an internal helper function in lines 65-77 that we will call once for each of three DBPedia entity types that we use in this example (people, organizations. and organizations).

Here we use spaCy so install the library and a small NLP model:

```
pip install import spacy
python -m spacy download en_core_web_sm
```

The agent custom tool example is short so I list the source file **custom_func_dbpedia.py** first and then we will dive into the code (edited to fit page width):

```
from QA import get_context_text

def get_context_data(query_text):
    """
       Method to get context text for entities from
       DBPedia using SPARQL query
    """

    query_text_data = get_context_text(query_text)
    return {"context_text": query_text_data}

## Custom function example using DBPedia

from typing import Type
from pydantic import BaseModel, Field
from langchain.tools import BaseTool

class GetContextTextFromDbPediaInput(BaseModel):
    """Inputs for get_context_data"""
```

```
20
21      query_text: str = \
22        Field(
23          description="query_text user supplied query text"
24        )
25
26  class GetContextTextFromDbPediaTool(BaseTool):
27      name = "get_context_data"
28      description =
29        """
30          Useful when you want to make a query and get
31          context text from DBPedia. You should enter
32          and text containing entity names.
33        """
34      args_schema: Type[BaseModel] = \
35            GetContextTextFromDbPediaInput
36
37      def _run(self, query_text: str):
38          text = get_context_data(query_text)
39          return text
40
41      def _arun(self, query_text: str):
42          raise NotImplementedError
43                 (
44                  "get_context_data does not support async"
45                 )
46
47  ## Create agent
48
49  from langchain.agents import AgentType
50  from langchain.chat_models import ChatOpenAI
51  from langchain.agents import initialize_agent
52
53  llm = ChatOpenAI(model="gpt-3.5-turbo-0613",
54                   temperature=0)
55
56  tools = [GetContextTextFromDbPediaTool()]
57
58  agent = initialize_agent(tools, llm,
59                           agent=AgentType.OPENAI_FUNCTIONS,
60                           verbose=True)
61
62  ## Run agent
```

```
63
64 agent.run(
65     """
66       What country is Berlin in and what other
67       information about the city do you have?
68     """
69 )
```

The class **GetContextTextFromDbPediaInput** defined in lines 18-24 defines a tool input variable with an English language description for the variable that the LLM can use. The class **GetContextTextFromDbPediaTool** defined in lines 26-45 defines the tool name, a description for the use of an LLM, and the definition of the required methon **_run**. Methon **_run** uses the utility finction **get_context_data** defined in the source file **QA.py**.

We define a GPT-3.5 model in lines 53-54. Out example only uses one tool (our custom tool). We define the tools list in line 56 and setup the agent in lines 58-60.

The example output is (edited to fit page width):

```
1  > Entering new AgentExecutor chain...
2
3  Invoking: `get_context_data` with `{'query_text':
4                                      'Berlin'}`
5
6  name='Berlin'
7  dbpedia_type='<http://dbpedia.org/ontology/Place>'
8
9   select distinct ?s ?comment where {
10     ?s
11       <http://www.w3.org/2000/01/rdf-schema#label>
12       'Berlin'@en .
13     ?s
14       <http://www.w3.org/2000/01/rdf-schema#comment>
15       ?comment  .
16     FILTER  (lang(?comment) = 'en') .
17     ?s
18       <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
19       <http://dbpedia.org/ontology/Place> .
20  } limit 15
21
22 {'context_text': "Berlin (/bɜːrˈlɪn/ bur-LIN, German: [bɛɐ̯ˈliːn]) is the capital and\
23  largest city of Germany by both area and population. Its 3.6 million inhabitants ma
24 ke it the European Union's most populous city, according to population within city l
25 imits. One of Germany's sixteen constituent states, Berlin is surrounded by the Stat
```

```
e of Brandenburg and contiguous with Potsdam, Brandenburg's capital. Berlin's urban
area, which has a population of around 4.5 million, is the second most populous urba
n area in Germany after the Ruhr. The Berlin-Brandenburg capital region has around 6
.2 million inhabitants and is Germany's third-largest metropolitan region after the
Rhine-Ruhr and Rhine-Main regions. . "}

Berlin is the capital and largest city of Germany. It is located in the northeastern\
 part of the country. Berlin has a population of approximately 3.6 million people, m
aking it the most populous city in the European Union. It is surrounded by the State
 of Brandenburg and is contiguous with Potsdam, the capital of Brandenburg. The urba
n area of Berlin has a population of around 4.5 million, making it the second most p
opulous urban area in Germany after the Ruhr. The Berlin-Brandenburg capital region
has a population of approximately 6.2 million, making it Germany's third-largest met
ropolitan region after the Rhine-Ruhr and Rhine-Main regions.

> Finished chain.
```

Another React Agent Tool Use Example: Search and Math Expressions

The example for this section is found in **langchain-book-examples/tool_search_math_example**.

In this chapter, we explore the development of a multi-tool intelligent agent by leveraging the power of LangChain and OpenAI's GPT models. With recent advancements in Large Language Models (LLMs) and agent frameworks, it's possible to create interactive tools capable of dynamic actions like web searches or evaluating mathematical expressions. This script demonstrates how to integrate two example tools, a DuckDuckGo-based search engine and a simple calculator, into a React-style agent capable of performing multi-step reasoning.

This example highlights the practicalities of creating a modular agent that combines natural language capabilities with functional tools. It illustrates the use of:

- LLMs for conversational abilities (via OpenAI's GPT-4 mini model).
- Third-party API tools, like DuckDuckGo search, for retrieving real-time information.
- Custom tools, such as a calculator, for handling specific operations.

By the end of this chapter, readers will understand how to define custom tools, set up a prompt template, and integrate these tools into an interactive agent capable of multi-step tasks. This example demonstrates the potential of tool-enhanced AI agents to solve both simple and complex problems dynamically.

Tools Example Implementation

```python
from langchain.agents import create_react_agent, Tool, AgentExecutor
from langchain.agents.agent_types import AgentType
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
import os

openai_key = os.getenv("OPENAI_API_KEY")
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.3, openai_api_key=openai_key)

ddg_api = DuckDuckGoSearchRun()

class DuckDuckGoSearchAPIWrapper:
    def __call__(self, query):
        results = ddg_api.invoke(query)
        #print(f"**** {results=}")
        return results if results else 'No results found'

class SimpleCalculator:
    def __call__(self, expression):
        try:
            return eval(expression)
        except Exception as e:
            return f"Error in calculation: {e}"

# Initialize the tools
search_tool = Tool(
    name="duckduckgo_search",
    func=DuckDuckGoSearchAPIWrapper(),
    description="Searches the web using DuckDuckGo"
)

calculator_tool = Tool(
    name="simple_calculator",
    func=SimpleCalculator(),
    description="Performs simple calculations"
)

# Define the tool chain
tools = [search_tool, calculator_tool]
tool_names = ["duckduckgo_search", "simple_calculator"]

agent_scratchpad = "thoughts: "
```

```
44
45 template = '''Answer the following questions as best you can. You have access to the\
46  following tools:
47
48 {tools}
49
50 Use the following format:
51
52 Question: the input question you must answer
53 Thought: you should always think about what to do
54 Action: the action to take, should be one of [{tool_names}]
55 Action Input: the input to the action
56 Observation: the result of the action
57 ... (this Thought/Action/Action Input/Observation can repeat N times)
58 Thought: I now know the final answer
59 Final Answer: the final answer to the original input question
60
61 Begin!
62
63 Question: {input}
64 Thought:{agent_scratchpad}'''
65
66 prompt = PromptTemplate.from_template(template)
67 print(prompt)
68
69 # Initialize the agent with tools
70 agent = create_react_agent(llm, tools, prompt)
71
72 agent_executor = AgentExecutor(agent=agent, tools=tools)
73
74 # Example #1 input for the chain
75 input_text = "search: What is the population of Canada?"
76
77 # Run the chain
78 result = agent_executor.invoke({"input": input_text, "chat_history": agent_scratchpa\
79 d})
80
81 # Print the result
82 print(result)
83
84 # Example #2 input for the chain
85 input_text = "calculator: 250 * 4"
86
```

```
# Run the chain
result = agent_executor.invoke({"input": input_text, "chat_history": agent_scratchpa\
d})

# Print the result
print(result)
```

The script imports key modules from LangChain, OpenAI, and a DuckDuckGo wrapper to build the agent framework. The OpenAI API key is pulled from an environment variable to maintain security.

Defining Tools

The agent is equipped with two tools:

- DuckDuckGo Search Tool: Uses the DuckDuckGoSearchRun API to fetch real-time web search results.
- Simple Calculator Tool: Evaluates mathematical expressions using Python's eval() function.

Each tool is encapsulated in its own class with a **call**() method to enable easy invocation.

The two tools are initialized using LangChain's Tool class. Each tool is assigned a name, func, and description.

A prompt template defines how the agent should format its responses. The agent follows a structured reasoning process, using thought-action-observation loops. Note that this example is written in a general purpose style supporting chain of thought and memory but here the tool use examples are one-shot, not multiple prompts.

Sample Output

```
$ python tool_search_math_example.py
{'input': 'search: What is the population of Canada?', 'chat_history': 'thoughts: ',\
 'output': 'The population of Canada is approximately 40,528,396 as of October 1, 20
23.'}
{'input': 'calculator: 250 * 4', 'chat_history': 'thoughts: ', 'output': '1000'}
```

LangChain Agent Tools Wrap Up

Writing custom agent tools is a great way to revisit the implementation of existing applications and improve them with the real world knowledge and reasoning abilities of LLMs. Both for practice in effectively using LLMs in your applications and also to extend your personal libraries and tools, I suggest that you look over you existing projects with an eye for either improving them using LLMs or refactoring them into reusable agent tools for your future projects.

Multi-prompt Search using LLMs, the Duckduckgo Search API, and Local Ollama Models

The short example we develop is inspired by commercial LLM apps like Perplexity[1]. I subscribe to the Perplexity Pro plan and find it useful so I wanted to implement my own simple minimalist Python library that provides the same type of multiple LLM pass on a query and search results, finally producing a relevant summary.

We will start by looking at example uses of this library and then, dear reader, you can decide if you want to hack on this example code and make it your own.

The example code uses three simple prompt templates used to filter out non-useful search results, to summarize the text from fetch search result web links, and to write a final summary:

```
prompt1 = "return concisely either 'Y' or 'N' if this query | %s | is matched well b\
y the following text: %s"
prompt2 = "Using the query | %s | summarize the following text including only materi\
al relevant to the query:\n%s"
prompt3 = "Using the query | %s | summarize in multiple paragraphs the following tex\
t including only material relevant to the query:\n%s"
```

Example 1: "Write a business plan for a new startup using LLMs and expertise in medical billing."

The example code has a ton of debug printout so here we only look at the final output summary:

[1] https://www.perplexity.ai

```
 Title: Business Plan for Mercury Concierge Medical Practice Startup

The business plan for the new startup, Mercury Concierge Medical Practice, is design\
ed to transition an existing medical practice to a concierge model, leveraging the e
xpertise of the Mercury Advisory Group. The primary objective is to provide personal
ized and comprehensive healthcare services while ensuring financial sustainability.

To achieve this, the startup will develop a mission statement, explore strategic opt\
ions, and create a budget. Key areas of focus include market entry strategy, operati
ons plan, HIM technology requirements, revenue management strategy, organizational d
esign, goal setting, and business unit strategies.

Mercury Concierge Medical Practice offers an initial flat-fee consultation service p\
riced at $3500. This service includes a full day of private coaching, Q&A sessions,
travel costs, a workbook, and a signed copy of the Handbook of Concierge Medical Pra
ctice Design.

Beyond the initial consultation, Mercury Advisory Group provides additional services\
 such as conducting SWOT (Strengths, Weaknesses, Opportunities, Threats), PEST (Poli
tical, Economic, Social, Technological) analyses, and Porter's Five Forces assessmen
ts to develop a business sustainability strategy. They also assist with the developm
ent of a lean/continuous improvement strategy, staffing plan, promotional strategy,
and operations plan.

The startup caters to various situations, including physicians who wish to jettison \
non-converting patients, those starting a new practice from scratch, or those transi
tioning while bound by contracts with health plans. The service is currently availab
le in the continental USA, with additional travel surcharges for Alaska, Guam, Puert
o Rico, Hawaii, and USVI.

In summary, Mercury Concierge Medical Practice aims to provide high-quality, persona\
lized healthcare services through a concierge model. By leveraging the expertise of
the Mercury Advisory Group, the startup will create a sustainable business strategy,
 develop an effective operations plan, and offer valuable consulting services to med
ical professionals.
```

Example 2: “Common Lisp and Deep Learning consultant”

Here we only look at the final output summary:

```
Mark Watson is a professional deep learning and artificial intelligence consultant w\
ho specializes in various programming languages such as Common Lisp, Clojure, Python
, Java, Haskell, and Ruby. He has authored over 20 books on AI, deep learning, and o
ther related topics, with his clients including notable companies like Google, Capit
al One, Disney, and Olive AI.

For those aiming to start with AI, Mark recommends several free courses: Generative \
AI by Google, AI for Beginners by Microsoft, and Artificial Intelligence (6.034) by
MIT. These courses cover topics such as generative AI, neural networks, deep learnin
g, computer vision, natural language processing, and more. Each course is text-based
 and offers exercises to help solidify understanding.
```

Example Code for Multi-prompt Search using LLMs, the Duckduckgo Search API, and Local Ollama Models

The example code script uses LangChain's Ollama interface, the Duckduckgo library for accessing Duckduckgo's internal quick results data (for low bandwidth non-commercial use), and the Trafilatura library for fetching p[lain text from a web URI:

```
from ddg import Duckduckgo
from langchain_community.llms.ollama import Ollama

# pip install llama-index html2text trafilatura
import trafilatura
from pprint import pprint

ddg_api = Duckduckgo()

llm = Ollama(
    model="mistral:v0.3",
    verbose=False,
)

prompt1 = "return concisely either 'Y' or 'N' if this query | %s | is matched well b\
y the following text: %s"
prompt2 = "Using the query | %s | summarize the following text including only materi\
al relevant to the query:\n%s"
prompt3 = "Using the query | %s | summarize in multiple paragraphs the following tex\
t including only material relevant to the query:\n%s"

def llm_search(query):
```

```
    results = ddg_api.search(query)
    data = results['data']
    good_results = []
    good_summaries = []
    for d in data:
        description = d['description']
        p = prompt1 % (query, description)
        s = llm.invoke(p)
        print(f"Prompt: {p}\nResponse: {s}\n\n")
        if s.strip()[0:1] == 'Y':
            good_results.append(d)
            uri = d['url']
            downloaded = trafilatura.fetch_url(uri)
            text = trafilatura.extract(downloaded)
            p2 = prompt2 % (query, text)
            s2 = llm.invoke(p2)
            good_summaries.append(s2)
    p3 = prompt3 % (query, "\n\n".join(good_summaries))
    final_summary = llm.invoke(p3)

    return (good_results, good_summaries, final_summary)
```

Here is an example use of this example code:

```
(results, summaries, final_summary) = llm_search("Write a business plan for a new st\
artup using LLMs and expertise in medical billing.")

print(f"\n\n****** Good Results ******\n\n")
print(results)

print(f"\n\n****** Good Summaries ******\n\n")
print(summaries)

print(f"\n\n****** Final Summary ******\n\n")
print(final_summary)
```

More Useful Libraries for Working with Unstructured Text Data

Here we look at examples using two libraries that I find useful for my work: EmbedChain and Kor.

EmbedChain Wrapper for LangChain Simplifies Application Development

Taranjeet Singh developed a very nice wrapper library EmbedChain https://github.com/embedchain/embedchain that simplifies writing "query your own data" applications by choosing good defaults for the LangChain library.

I will show one simple example that I run on my laptop to search the contents of all of the books I have written as well as a large number of research papers. You can find my example in the GitHub repository for this book in the directory **langchain-book-examples/embedchain_test**. As usual, you will need an OpenAI API account and set the environment variable **OPENAI_API_KEY** to the value of your key.

I have copied PDF files for all of this content to the directory **~/data** on my laptop. It takes a short while to build a local vector embedding data store so I use two Python scripts. The first script **process_pdfs.py** that is shown here:

```
# reference: https://github.com/embedchain/embedchain

from embedchain import App
import os

test_chat = App()

my_books_dir = "/Users/mark/data/"

for filename in os.listdir(my_books_dir):
    if filename.endswith('.pdf'):
        print("processing filename:", filename)
        test_chat.add("pdf_file",
                      os.path.join(my_books_dir,
                      filename))
```

Here is a demo Python script **app.py** that makes three queries:

```
from embedchain import App

test_chat = App()

def test(q):
    print(q)
    print(test_chat.query(q), "\n")

test("How can I iterate over a list in Haskell?")
test("How can I edit my Common Lisp files?")
test("How can I scrape a website using Common Lisp?")
```

The output looks like:

```
$ python app.py
How can I iterate over a list in Haskell?
To iterate over a list in Haskell, you can use recursion or higher-order functions l\
ike `map` or `foldl`.

How can I edit my Common Lisp files?
To edit Common Lisp files, you can use Emacs with the Lisp editing mode. By setting \
the default auto-mode-alist in Emacs, whenever you open a file with the extensions "
.lisp", ".lsp", or ".cl", Emacs will automatically use the Lisp editing mode. You ca
n search for an "Emacs tutorial" online to learn how to use the basic Emacs editing
commands.

How can I scrape a website using Common Lisp?
One way to scrape a website using Common Lisp is to use the Drakma library. Paul Nat\
han has written a library using Drakma called web-trotter.lisp, which is available u
nder the AGPL license at articulate-lisp.com/src/web-trotter.lisp. This library can
be a good starting point for your scraping project. Additionally, you can use the wg
et utility to make local copies of a website. The command "wget -m -w 2 http:/knowle
dgebooks.com/" can be used to mirror a site with a two-second delay between HTTP req
uests for resources. The option "-m" indicates to recursively follow all links on th
e website, and the option "-w 2" adds a two-second delay between requests. Another o
ption, "wget -mk -w 2 http:/knowledgebooks.com/", converts URI references to local f
ile references on your local mirror. Concatenating all web pages into one file can a
lso be a useful trick.
```

Kor Library

The Kor library was written by Eugene Yurtsev. Kor is useful for using LLMs to extract structured data from unstructured text. Kor works by generating appropriate prompt text to explain to GPT-3.5 what information to extract and adding in the text to be processed.

The GitHub repository for Kor[1] is under active development so please check the project for updates. Here is the documentation[2].

For the following example, I modified an example in the Kor documentation for extracting dates in text.

```
" From documentation: https://eyurtsev.github.io/kor/"

from kor.extraction import create_extraction_chain
from kor.nodes import Object, Text, Number
from langchain.chat_models import ChatOpenAI
from pprint import pprint
import warnings ; warnings.filterwarnings('ignore')

llm = ChatOpenAI(
    model_name="gpt-3.5-turbo",
    temperature=0,
    max_tokens=2000,
    frequency_penalty=0,
    presence_penalty=0,
    top_p=1.0,
)

schema = Object(
    id="date",
    description=(
        "Any dates found in the text. Should be output in the format:"
        " January 12, 2023"
    ),
    attributes = [
        Text(id = "month",
             description = "The month of the date",
             examples=[("Someone met me on December 21, 1995",
                        "Let's meet up on January 12, 2023 and discuss our yearly bu\
dget")])
```

[1] https://github.com/eyurtsev/kor
[2] https://eyurtsev.github.io/kor/

```
    ],
)

chain = create_extraction_chain(llm, schema, encoder_or_encoder_class='json')


pred = chain.predict_and_parse(text="I will go to California May 1, 2024")['data']
print("* month mentioned in text=", pred)
```

Sample output:

```
$ python dates.py
* month mentioned in text= {'date': {'month': 'May'}}
```

Kor is a library focused on extracting data from text. You can get the same effects by writing for own prompts manually for GPT style LLMs but using Tor can save development time.

Book Wrap Up

This book has been fun to write but it has also somewhat frustrating.

It was fun because I have never been as excited by new technology as I have by LLMs and utility software like LangChain and LlamaIndex for building personalized applications.

This book was frustrating in the sense that it is now so very easy to build applications that just a few years would have been impossible to write. Usually when I write books I have two criteria: I only write about things that I am personally interested in and use, and I also hope to figure out non-obvious edge cases and make easier for my readers to use new tech. Here my frustration is writing about something that it is increasingly simple to do so I feel like my value is diminished.

All that said I hope, dear reader, that you found this book to be worth your time reading.

What am I going to do next? Although I am not fond of programming in JavaScript (although I find TypeScript to be somewhat better), I want to explore the possibilities of writing an open source Persistent Storage Web App Personal Knowledge Base Management System. I might get pushback on this but I would probably make it Apple Safari specific so I can use Apple's CloudKit JS to make its use seamless across macOS, iPadOS, and iOS. If I get the right kind of feedback on social media I might write a book around this project.

Thank you for reading my book!

Best regards, Mark Watson